

Major Project Report

On

FUTURISTIC EV INFRASTRUCTURE-SLOT RESERVATION

**Submitted in partial fulfilment of the requirement
For the award of the degree of**

Bachelor of Technology

In

Department of Electronics and Communication Engineering

By

B.HEMENDRASIVA SAI - 212U1A0412

K.PRAVEEN KUMAR - 212U1A0441

K.SUMANTH - 212U1A0448

Under the Esteemed Guidance of

Mr. B. Suresh Kumar M.tech, Asst. Professor, Dept of ECE



**DEPARTMENT OF ELECTRONICS & COMMUNICATION ENGINEERING
GEETHANJALI INSTITUTE OF SCIENCE & TECHNOLOGY**

A Unit of USHODAYA EDUCATIONAL SOCIETY

(Approved by AICTE, New Delhi & Permanently Affiliated to JNTUA, Anantapuramu)

(Autonomous)

Accredited by NAAC-‘A’ Grade & NBA

An ISO 9001:2015 certified Institution: Recognized under Sec. 2(f) of UGC Act, 1956 3rd Mile,
Bombay Highway, Gangavaram (V), Kovur(M), SPSR Nellore (Dt), Andhra Pradesh, India-
524137 Ph. No. 08622-212769, E-Mail: geethanjali@gist.edu.in, Website: www.gist.edu.in

(Batch: 2021-25)



GEETHANJALI INSTITUTE OF SCIENCE AND TECHNOLOGY

DEPARTMENT OF ELECTRONICS AND COMMUNICATION ENGINEERING

Institute Vision:

To emerge as a center of technical excellence, transforming the engineering aspirants into dynamic and socially responsible technocrats.

Institute Mission:

IM1 Implementing effective strategies for imparting quality education in a conducive academic ambience to upgrade the intellectual and professional dimensions of the learner's personality.

IM2 Facilitating skill development and research to fulfil societal needs.

IM3 Inculcating moral principles, environmental consciousness and social responsibility among students.

IM4 Grooming the students to handle the career challenges successfully.

Department Vision:

Achieving academic excellence in Electronics and Communication Engineering by shaping next- generation technocrats keeping pace with socio-economic needs.

Department Mission:

DM1 Adopting outcome-oriented teaching-learning processes to provide comprehensive knowledge in the application of Electronics and Communication Engineering principles.

DM2 Striving for implementation of advanced technology to cater to industrial demands and societal concerns

DM3 Producing highly skilled and responsible professionals with robust ethical values

DM4 Integrating technical capabilities, life skills and entrepreneurship abilities to produce dynamic contributors to social advancement

Program Educational Objectives:

Graduates of B.Tech in Electronics and Communication Engineering Program shall be able to

PEO1: Demonstrating a deep passion for continuous learning through technical expertise for a promising career

PEO2: Exhibiting a strong commitment to serving the society with adherence to professional ethics

PEO3: Managing resources efficiently as competent engineers through effective social interaction

PEO4: Engaging in advanced learning and contributing to technological innovations

Program Specific Outcomes:

PSO1: Professional Skills: Design and develop electronic circuits and communication systems, applying the principles of signal, image processing, VLSI, Embedded and wireless applications relevant to industry and society

PSO2: Software Knowledge: Adopting software tools like Matlab, Xilinx, Microwind, NS-2 to develop intelligent systems to offer customized solutions.

Major Project Report

On

FUTURISTIC EV INFRASTRUCTURE-SLOT RESERVATION

Submitted in partial fulfilment of the requirement
For the award of the degree of

Bachelor of Technology

In

Department of Electronics and Communication Engineering

By

B.HEMENDRASIVA SAI - 212U1A0412

K.PRAVEEN KUMAR - 212U1A0441

K.SUMANTH - 212U1A0448

Under the Esteemed Guidance of

Mr. B. Suresh Kumar M.tech, Asst. Professor, Dept of ECE



DEPARTMENT OF ELECTRONICS & COMMUNICATION ENGINEERING

GEETHANJALI INSTITUTE OF SCIENCE & TECHNOLOGY

A Unit of USHODAYA EDUCATIONAL SOCIETY

(Approved by AICTE, New Delhi & Permanently Affiliated to JNTUA, Anantapuramu)

(Autonomous)

Accredited by NAAC-‘A’ Grade & NBA

An ISO 9001:2015 certified Institution: Recognized under Sec. 2(f) of UGC Act, 1956 3rd Mile,
Bombay Highway, Gangavaram (V), Kovur(M), SPSR Nellore (Dt), Andhra Pradesh, India-
524137 Ph. No. 08622-212769, E-Mail: geethanjali@gist.edu.in, Website: www.gist.edu.in

(Batch: 2021-25)

**DEPARTMENT OF ELECTRONICS & COMMUNICATION
ENGINEERING**

GEETHANJALI INSTITUTE OF SCIENCE & TECHNOLOGY

A Unit of USHODAYA EDUCATIONAL SOCIETY
(Approved by AICTE, New Delhi & Permanently Affiliated to JNTUA, Anantapuramu)
(Autonomous)

Accredited by NAAC-'A' Grade & NBA

An ISO 9001:2015 certified Institution: Recognized under Sec. 2(f) of UGC Act, 1956
3rd Mile, Bombay Highway, Gangavaram (V), Kovur(M), SPSR Nellore (Dt), Andhra Pradesh, India- 524137
Ph. No. 08622-212769, E-Mail: geethanjali@gist.edu.in, Website: www.gist.edu.in



CERTIFICATE

This is to certify that the project report entitled “**FUTURISTIC EV INFRASTRUCTURE-SLOT RESERVATION**” that is being submitted by **B.HEMENDRA SIVA SAI, K.PRAVEEN KUMAR, K.SUMANTH** in partial fulfilment for the award of the Degree of **Bachelor of Technology in Electronics and Communication Engineering** to the Jawaharlal Nehru Technological University Anantapur is a record of bonafide work carried out under my guidance and supervision.

Internal Guide
Mr. B. Suresh Kumar,
M.Tech, Asst. Professor

Head of the Department
Dr. Usthulamuri Penchalaiah,
Professor & Head

Submitted for University Examination (Viva Voce) held on _____

Internal Examiner

External Examiner

(Batch: 2021-25)

ACKNOWLEDGEMENT

We express our deepest sense of gratitude to **Sri. N. Sudhakar Reddy, B. Tech, SECRETARY and CORRESPONDENT**, Geethanjali Institute of Science and Technology, Nellore and other members of Management for providing all the facilities for this work.

Our special thanks to our respectable Director **Prof. Dr. G. Subba Rao, M.E., Ph.D., SMIEE., LMISTE.,** Geethanjali Institute of Science and Technology, Nellore, for his continuous support during the project work.

We express our sincere thanks to our beloved Principal **Dr. Sundeep Kumar K, M.E, Ph.D.,** Geethanjali Institute of Science and Technology, Nellore, for his consistent help and valuable suggestions.

We express our sincere thanks to our HoD, **Dr. Usthulamuri Penchalaiah, M.Tech., Ph.D.,** Geethanjali Institute of Science and Technology, Nellore, for his timely suggestions and help during the progress of project work.

We would express heartfelt gratitude and indebtedness to our guide, **Mr. B. Suresh Kumar, M.Tech. Asst. professor, Dept. of ECE,** Geethanjali Institute of Science and Technology, Nellore, for his keen interest, critical, constructive and support each and every stage of project and for giving his advice when ever needed.

During the entire course of dissertation work we received valuable academic inputs as well as moral support from other departments, general teaching and nonteaching faculty at **Geethanjali Institute of Science and Technology**, Nellore. We were motivated by the uphold and moral encouragement given to me by my beloved parents. Finally, we feel to extend our sincere thanks to those who helped directly and indirectly during completion of our project work.

Project Associates:

B. HEMENDRA SIVA SAI	(212U1A0412)
K. PRAVEEN KUMAR	(212U1A0441)
K.SUMANTH	(212U1A0448)

ABSTRACT

The rise of electric vehicles (EVs) is driving the need for an efficient charging infrastructure to support their growing adoption. However, existing charging stations lack a reservation system, forcing users to check availability manually, leading to long wait times and inefficiencies. This system introduces an automated EV slot reservation solution that allows users to pre-book charging slots remotely, enhancing accessibility and convenience. It integrates IoT devices for real-time monitoring, an RTC (Real-Time Clock) module for accurate scheduling, GSM for remote communication, and an LCD display for user interaction. By providing real-time slot availability and remote booking options, the system ensures efficient station management, reduces congestion, and improves the overall charging experience and security.

CONTENTS

CHAPTER NO	DESCRIPTION	PAGE NO
1.	Introduction	1- 6
	1.1 Overview	2
	1.2 Research Motivation	2 - 3
	1.3 Existing Systems	3
	1.4 Problem Statement	3 - 4
	1.5 Research Objective	4
	1.6 Applications	4 - 5
	1.7 Organization of Theisis	5 - 6
2.	Literature Survey	7 - 11
	2.1 Related Works	8 – 11
3.	Existing System	12 - 14
	3.1 Existing methods	13 - 14
4.	Proposed System	15 - 23
	4.1 Introduction	16
	4.2 Block Diagram	16 - 17
	4.3 Project working	17 - 19

4.4 Schematic Diagram	19 - 20
4.5 Advantages	20 - 21
4.6 Applications	22 -23
5. Embedded Systems	24 - 36
5.1 What are Embedded Systems	25 - 26
5.1.1 History	26 - 27
5.1.2 Tools	27
5.1.3 Resources	27 - 28
5.1.4 Real Time Issues	28
5.2 Need for Embedded Systems	28
5.2.1 Debugging	28 – 29
5.2.2 Reliability	29 - 30
5.3 Explanation of Embedded Systems	30 - 34
5.3.1 Software Architecture	30 -32
5.3.2 Stand Alone Embedded System	32
5.3.3 Real Time Embedded Systems	32 - 33
5.3.4 Network Communication ES	33
5.3.5 Different types of processing units	34

5.4 Applications of Embedded Systems	34 – 36
5.4.1 Consumer applications	34
5.4.2 Office Automation	34 – 35
5.4.3 Industrial Automation	35
5.4.4 Computer Networking	35 – 36
5.4.5 Tele Communication	36
6. Hardware Description	37 – 64
6.1 ESP32 Controller	38 – 45
6.1.1 ESP32 Features and Specifications	41 – 43
6.1.2 ESP32 Functions	43 – 45
6.2 Regulated Power Supply	45 – 51
6.2.1 Bridge Full Wave Rectifier	47 – 49
6.2.2 Voltage Regulation	49 – 50
6.2.3 Resistor and LED	50 – 51
6.3 Global System for Mobile communications	51 – 55
6.3.1 Definition	51 – 52
6.3.2 Modem Specifications	52 – 53
6.3.3 General Features	53 – 55

FUTURISTIC EV INFRASTRUCTURE-SLOT RESERVATION

	6.4 Buzzer	55 – 57
	6.5 Relay	57 – 58
	6.5.1 Relay Driver	58
	6.6 LCD Display	58 – 62
	6.6.1 LCD Background	58 – 60
	6.6.2 SETB RW	60
	6.6.3 CLR EN	60 -61
	6.6.4 Applications	61 – 62
	6.7 Real Time Clock	62 – 63
	6.7.1 Functions	62 – 63
	6.7.2 Key Features	63
	6.8 Keypad	63 – 64
7.	Software Description	65 - 69
	7.1 Arduino IDE- Compiler	65 – 69
8.	Code	70 – 80
9.	Results	81 – 84
10.	Conclusion	85 – 86
	REFERENCES	87 – 90

LIST OF FIGURES

FIGURE NO	FIGURE NAME	PAGE NO
4.1	Block Diagram	16
4.4	Schematic Diagram	19
5.1	A Modern Example of Embedded System	26
5.2	Network Communication Embedded System	33
5.3	Automatic Coffee Makes Equipment	34
5.4	Fax Machine	35
5.5	Printing Machine	35
5.6	Robot	35
5.7	Computer Networking	36
5.8	Cell Phone	36
5.9	Web Camera	36
6.1	ESP32 Controller	38
6.2	ESP32 Features and Specifications	41
6.8	Block Diagram of Power Supply	45

FUTURISTIC EV INFRASTRUCTURE-SLOT RESERVATION

6.9	Power Supply Circuit Diagram	45
6.10	Regulated Power Supply	46
6.11	Circuit Diagram of RPS with LED	46
6.12	Bridge Rectifier and Full Wave Rectifier	47
6.13	DB107	48
6.14	Construction of a Capacitor	49
6.15	Electrolytic	49
6.16	VoltageRegulator	50
6.18	Resistor	51
6.19	Color Bands In Resistor	51
6.20	Inside a LED	51
6.21	Parts of a LED	51
6.3	GSM Module	52
6.4	Buzzer	55
6.41	Circuit of Buzzer	57
6.5	Circuit Of Relay	57
6.51	Relay Driver	57
6.61	LCD Pin Diagram	59

FUTURISTIC EV INFRASTRUCTURE-SLOT RESERVATION

6.62	LCD Schematic	59
6.7	RTC Module	62
6.8	Custom Keyboard	64
9.1	Hardware Equipment	82
9.2	Password Entry	82
9.3	Commands	83
9.4	Website For Remote Monitoring	83

FUTURISTIC EV INFRASTRUCTURE-SLOT RESERVATION

ABSTRACT

The rise of electric vehicles (EVs) is driving the need for an efficient charging infrastructure to support their growing adoption. However, existing charging stations lack a reservation system, forcing users to check availability manually, leading to long wait times and inefficiencies. This system introduces an automated EV slot reservation solution that allows users to pre-book charging slots remotely, enhancing accessibility and convenience. It integrates IoT devices for real-time monitoring, an RTC (Real-Time Clock) module for accurate scheduling, GSM for remote communication, and an LCD display for user interaction. By providing real-time slot availability and remote booking options, the system ensures efficient station management, reduces congestion, and improves the overall charging experience and security.

CHAPTER – 1
INTRODUCTION

1.1 Overview:

The global transportation sector is undergoing a significant transformation, marked by the increasing shift from conventional fuel-powered vehicles to electric vehicles (EVs). This shift is driven by the need to reduce greenhouse gas emissions, mitigate fossil fuel depletion, and promote sustainable mobility solutions. The transportation industry is a major contributor to environmental pollution, and governments worldwide are implementing initiatives such as subsidies, incentives, and infrastructure development to encourage the adoption of EVs.

However, despite the advantages of EVs, challenges persist in managing charging infrastructure. Unlike traditional fuel stations, EV charging stations require longer durations for recharging, resulting in long waiting times, underutilized infrastructure, and user inconvenience. Range anxiety—the fear of battery depletion before locating a charging station—further complicates the adoption of EVs.

To address these issues, an intelligent EV Slot Reservation System is proposed. This system integrates emerging technologies such as IoT, cloud computing, and AI to ensure optimal charging station utilization, minimize waiting times, improve user experience, and promote sustainable energy management through smart load balancing.

1.2 Research Motivation

- The motivation behind the development of the EV Slot Reservation System stems from the increasing need to overcome challenges associated with the current EV charging framework. Present systems predominantly follow a first-come, first-served approach, resulting in unpredictable waiting times, inefficient station utilization, and dissatisfaction among users.
- The growing demand for EVs requires a structured and scalable charging management system. By incorporating IoT-based sensors, AI-powered algorithms, and cloud infrastructure, the proposed system offers dynamic slot allocation, real-time monitoring, and seamless user interaction. It improves operational efficiency, enhances convenience,

reduces grid overloading, and supports smart city initiatives by integrating sustainable energy practices.

1.3 Existing Systems

Existing EV charging systems largely rely on manual processes or basic automated mechanisms, leading to multiple limitations:

- **First-Come, First-Served Basis:** Causes congestion, unpredictable waiting times, and inconvenience.
- **Lack of Real-Time Slot Updates:** Users often waste time and effort searching for available slots.
- **Inefficient Resource Utilization:** Charging stations experience idle periods or overloading without proper management.
- **Manual Payment Systems:** Payment processing lacks integration with modern digital platforms, reducing user convenience.
- **Limited Scalability:** Existing systems are not designed to accommodate increasing demand or integrate new technologies.

These limitations highlight the need for an intelligent, automated, and secure slot reservation system capable of enhancing efficiency and scalability

1.4 Problem Statement

While various EV charging stations have been introduced, they are constrained by the absence of effective slot management mechanisms. Key issues include:

- Unpredictable waiting times due to lack of pre-booking options.
- Inefficient station utilization, leading to congestion and idle periods.

- Grid overloading during peak usage without proper load balancing.
- Lack of real-time updates and transparency in slot availability
- Security concerns such as unauthorized usage and fraudulent bookings

The proposed system aims to eliminate these challenges by incorporating smart technologies for efficient resource allocation, real-time monitoring, and enhanced user satisfaction

1.5 Research Objective

The primary objective of this research is to design and implement an efficient IoT-based EV Slot Reservation System. The system aims to:

- Enable users to pre-book slots through a user-friendly mobile or web platform.
- Integrate IoT-based sensors to monitor real-time station occupancy.
- Implement a cloud-based database to manage reservations and historical data.
- Employ AI-driven algorithms for dynamic slot allocation based on demand and availability.
- Facilitate seamless and secure cashless payment options.
- Support scalability and integration with smart city infrastructure and renewable energy sources.

1.6 Applications

The proposed EV Slot Reservation System can be deployed across multiple real-world scenarios:

- **Public Charging Stations:** Reduces congestion and improves user convenience in urban areas.
- **Corporate EV Fleets:** Optimizes charging schedules, reducing downtime and improving fleet efficiency.
- **Highway Charging Stations:** Enables long-distance travelers to plan their charging stops in advance, reducing range anxiety.
- **Smart City Integration:** Easily scalable and adaptable to future smart city advancements, including renewable energy integration.

1.7 Organization of the thesis

Chapter 1 Chapter 1 introduces the research topic, presenting the motivation, challenges, research objectives, and applications of the EV Slot Reservation System.

Chapter 2 provides a comprehensive literature survey, reviewing related studies and technologies relevant to EV charging systems and slot reservation frameworks.

Chapter 3 discusses the existing systems in detail, outlining their limitations and the need for advanced solutions.

Chapter 4 elaborates on the proposed system architecture, explaining the role of each hardware and software component involved in the system design.

Chapter 5 focuses on the implementation process, including hardware-software integration, user interface development, and the deployment of IoT and AI technologies.

Chapter 6 details the software environment used, such as programming languages, frameworks, and libraries essential for the development of the system.

Chapter 7 presents the source code of the system, providing insights into the coding structure,

functions, and logic used in implementing the system.

Chapter 8 discusses the experimental results and evaluation of the system's performance. It includes analysis based on key metrics such as station utilization, user convenience, and response time.

Chapter 9 concludes the thesis, summarizing key findings and contributions. It also outlines potential future enhancements, including scalability, renewable energy integration.

CHAPTER-2
LITERATURE SURVEY

2.1 Related works

Zhang et al. (2021) developed an intelligent EV charging slot reservation system that leverages real-time traffic data and energy demand forecasting to optimize charging station usage. Their system dynamically analyzes traffic conditions and predicts energy demand to pre-allocate charging slots efficiently. The model improves overall charging efficiency, minimizes user waiting times, and reduces the occurrence of underutilized stations. By incorporating predictive analytics, the system ensures a seamless EV charging experience and contributes to the smooth operation of EV networks **【1】** .

Kumar et al. (2020) proposed an IoT-enabled EV slot reservation framework integrated with cloud computing technology. Their system facilitates real-time slot booking by connecting IoT sensors at charging stations to a centralized cloud database. This approach ensures that users receive immediate updates on slot availability, while also optimizing energy consumption based on usage patterns. The research demonstrates how cloud-based reservation systems can enhance user convenience, reduce operational costs, and improve resource utilization across charging networks **【2】** .

Lee and Park (2022) explored blockchain implementation in EV charging slot reservation systems to strengthen security and prevent fraudulent reservations. Their decentralized framework employs smart contracts to ensure transparent, tamper-proof transactions. This approach not only enhances data integrity but also protects against unauthorized slot bookings. The research shows that integrating blockchain technology into EV slot reservation systems effectively addresses trust and security concerns while maintaining operational efficiency **【3】** .

Chen et al. (2021) developed an AI-driven predictive model for charging slot allocation. Their system utilizes historical demand patterns combined with real-time data to anticipate peak usage times and allocate slots accordingly. By employing machine learning algorithms, the model continuously improves its predictions over time. The study highlights how predictive analytics can significantly enhance the efficiency of EV charging networks and reduce user wait times **【4】** .

Singh et al. (2019) presented a dynamic pricing model designed to regulate EV charging demand. The pricing strategy adjusts slot rates based on real-time demand fluctuations and congestion levels, encouraging users to utilize off-peak hours. This approach effectively balances the load on charging stations, promotes energy efficiency, and prevents grid overloading. The research demonstrates how dynamic pricing mechanisms can optimize resource distribution and improve user behavior **【5】** .

Patel et al. (2020) proposed a cloud-based scheduler for EV charging stations. The system manages slot reservations by processing real-time user requests alongside station availability data. The scheduler allocates charging slots dynamically, ensuring fair load distribution and preventing overutilization of specific stations. The research emphasizes how cloud integration and real-time scheduling can improve service reliability and user satisfaction **【6】** .

Wang et al. (2021) designed an optimization framework that incorporates power grid constraints into EV slot reservation systems. Their model takes into account available grid capacity, energy demand, and user requests to distribute charging slots effectively. This approach minimizes grid imbalance and improves station efficiency. The research suggests that such integrated frameworks are crucial to ensuring reliable, scalable EV charging networks **【7】** .

Li et al. (2022) integrated EV slot reservation systems into smart grids to balance electricity loads and prevent network congestion. Their system allows coordinated charging by synchronizing reservations with real-time grid conditions. This coordination reduces stress on power distribution systems, supports sustainable energy use, and enables better load management. The research underscores the importance of smart grid integration in modern EV infrastructure **【8】** .

Gomez et al. (2019) introduced a Vehicle-to-Grid (V2G)-enabled slot reservation system, where EVs not only consume energy but can also supply power back to the grid during peak hours. Their system supports bidirectional energy flow and incorporates pre-reserved slots to manage power transactions. This enhances grid stability and allows EV owners to benefit from energy trading. The study demonstrates how V2G technology can transform EVs into active participants in energy management **【9】**

Kumar and Singh (2020) developed an algorithm for real-time slot allocation based on user

preferences and station availability. Their system dynamically adjusts bookings, minimizing wait times and ensuring equitable distribution of charging slots. The research shows that real-time algorithms can improve the efficiency of charging station operations and enhances user experience 【10】 .

Smith et al. (2018) presented a mobile application that allows users to reserve EV charging slots in advance. The app provides real-time updates, estimated waiting times, payment integration, and navigation support. Their solution addresses range anxiety and user inconvenience by offering seamless access to charging infrastructure. The research highlights the role of mobile-based platforms in improving user engagement and station efficiency 【11】 .

Zhou and Fang (2021) proposed a multi-agent system to automate EV slot reservations. Independent agents represent user demands and station capacities, coordinating dynamically to distribute charging slots fairly. Their system enhances resource allocation efficiency and prevents over-concentration at specific stations. The research illustrates how multi-agent coordination can improve scalability and fairness in EV charging networks 【12】 .

Gupta et al. (2020) studied demand-side management techniques to optimize charging slot allocation. Their approach involves shifting charging loads to off-peak hours by offering incentives and dynamically adjusting slot availability. This reduces peak-hour congestion, prevents grid overloads, and lowers energy costs. The research emphasizes how demand-side management can contribute to sustainable EV infrastructure 【13】

Kim et al. (2019) employed predictive analytics to estimate future charging slot availability. Their model uses historical data trends and external factors, such as weather conditions, to forecast slot usage. This foresight allows users to plan their trips effectively, ensuring that charging stations are available when required. The research shows the benefits of incorporating predictive tools into reservation systems 【14】

Alam et al. (2021) implemented a machine learning-based EV slot reservation model that learns from user behaviors and station utilization patterns. defines its algorithms to minimize user wait times and improve efficiency. Their research demonstrates how adaptive machine learning models can optimize reservation processes in real-time

【15】 .

Hernandez et al. (2020) proposed an RFID-based system for automating EV charging slot bookings and access control. Users authenticate themselves using RFID tags, and the system manages reservations and station access accordingly. This contactless solution enhances security, speeds up the booking process, and prevents unauthorized station usage. The research highlights how RFID integration simplifies the user experience 【16】 .

Nakamura et al. (2021) integrated car-sharing services with EV slot reservation systems to ensure optimal utilization of charging infrastructure. Their approach allows shared EV fleets to access pre-designated slots, improving overall station throughput. The research suggests that integrating shared mobility services with slot management supports sustainable urban transportation solutions 【17】 .

Brown et al. (2022) analyzed how reservation systems impact the profitability of public EV charging stations. Their study shows that efficient reservation mechanisms increase station utilization rates, optimize revenue generation, and improve customer satisfaction. The research highlights the economic benefits of structured slot reservation systems 【18】 .

Liu et al. (2020) proposed a communication protocol between EV users and smart grids for efficient slot reservation and energy management. The protocol facilitates real-time data exchange, allowing dynamic load balancing and enhanced grid resilience. Their research emphasizes the importance of communication infrastructure in managing large-scale EV networks 【19】 .

Rossi et al. (2021) addressed challenges in EV slot reservation in urban areas characterized by high population density and limited infrastructure. They proposed solutions such as smart zoning, priority-based allocation, and real-time monitoring to alleviate congestion. Their study suggests innovative strategies to improve accessibility and manage demand in densely populated cities 【20】 .

CHAPTER – 3
EXISTING SYSTEM

3.1 Existing Methods

Existing EV charging systems primarily rely on conventional methods for managing slot availability, resource allocation, and user access. While these systems have facilitated the initial adoption of electric vehicles, they present several limitations in terms of scalability, efficiency, and user convenience. This chapter discusses the various components and features typically found in existing EV charging infrastructures:

Slot Allocation Mechanism

Most current EV charging stations operate on a first-come, first-served basis. Users arrive at the station and occupy available slots without any prior booking. This often leads to long waiting times, inefficient utilization of charging points, and increased user dissatisfaction, especially during peak hours.

Physical Charging Infrastructure

The physical setup of EV charging stations includes standard charging equipment and connectors compatible with various EV models. While the hardware is generally robust, it lacks intelligent management capabilities such as real-time monitoring and dynamic allocation, resulting in underutilized or congested stations.

Payment Systems

Traditional payment systems at EV stations involve manual or basic automated transactions through debit/credit cards or prepaid RFID cards. These systems often lack integration with digital wallets, secure authentication, or seamless mobile app interfaces, reducing payment flexibility and user convenience.

CCTV Surveillance

Many EV charging stations deploy closed-circuit television (CCTV) cameras to monitor station activities. While surveillance provides a basic level of security, existing systems typically lack advanced analytics capabilities such as real-time anomaly detection, leading to limited effectiveness in addressing misuse or unauthorized access.

Access Control Systems

Access to charging stations is often unregulated or managed via simple authentication mechanisms like RFID cards. In some cases, users may not require any form of identity verification, making the system vulnerable to unauthorized use or fraudulent activities.

Security Personnel

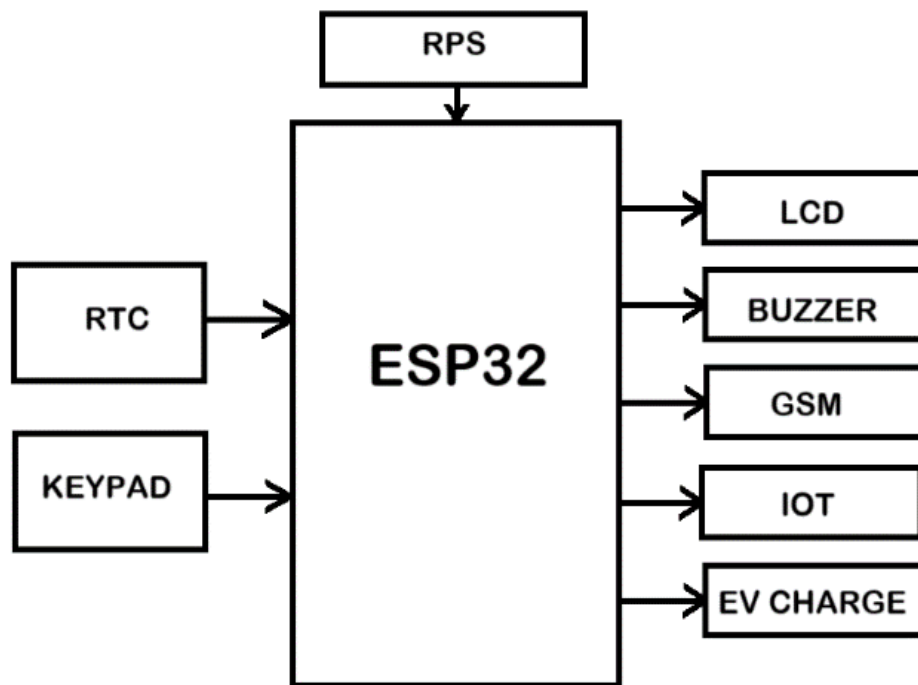
Certain high-traffic charging hubs may employ security personnel to monitor the premises and address any disputes or emergencies. However personnel alone does not guarantee efficient slot management or prevention of unauthorized access.

CHAPTER – 4
PROPOSED SYSTEM

4.1 Introduction

In recent years, the application of deep learning techniques, particularly Convolutional Neural Networks (CNNs), has revolutionized various domains, including image classification. One of the fascinating areas where CNNs have demonstrated remarkable performance is in the classification of larvae images. Larvae, being a crucial stage in the life cycle of many organisms, pose unique challenges for classification due to their diverse morphological features and subtle differences between species. Accurate and efficient classification of larvae images holds immense significance in various fields, including biology, ecology, agriculture, and environmental science.

4.2 BLOCK DIAGRAM



The working of the EV Slot Reservation System is explained here. During the initial setup, the system is programmed to monitor charging slot availability and store real-time data on a cloud server. The user interacts with the system using a custom-designed 3-key keypad module. The keypad consists of three functional keys: Increment (I), Decrement (D), and Enter (E), which are programmed to navigate slot options and confirm bookings.

When the user arrives at the charging station, they first use the keypad to browse through the available charging slots displayed on the 16x2 LCD monitor. The Increment key allows the user to scroll forward through the slots, while the Decrement key enables backward navigation. Each slot's current status (Available/Occupied) is clearly displayed on the LCD.

Once the user identifies an available slot, they press the Enter key to initiate the reservation process. The ESP32 microcontroller, acting as the core controller, processes this input and communicates with the cloud database through its built-in Wi-Fi module. The system checks the slot's availability status in real-time, ensuring no conflicts in the reservation process.

If the selected slot is confirmed to be available, the system proceeds to update the reservation details on the cloud server. The LCD monitor displays a booking confirmation message to the user, and the buzzer generates a beep sound to notify successful slot reservation.

Simultaneously, the DC motor simulates the physical control of the charging slot—this could represent opening a gate or activating a charging point, depending on the physical setup. The motor is triggered to operate only after successful reservation confirmation.

If the selected slot is found to be already booked or unavailable, the system promptly notifies the user via an alert message on the LCD and an audio signal from the buzzer, prompting them to select a different slot.

Throughout the process, all events, including slot selection, reservation confirmation, and system status updates, are displayed on the LCD for user guidance. The system maintains a real-time audit trail of all bookings and slot usage, synchronized with the cloud server for remote monitoring and analysis.

Additionally, the system ensures that all interactions are handled securely. Timeouts may be incorporated to prevent unauthorized access or accidental misuse. In case of repeated failed reservation attempts, system logs can be reviewed for security audits or troubleshooting.

This structured working approach ensures efficient slot allocation, real-time monitoring, and enhanced user convenience, thereby optimizing the utilization of EV charging stations.

4.3 Project Working:

In this system, there are totally five Sections:

- 1.Regulated Power Supply [RPS]
- 2.Input Section

- RTC
- Keypad
- IoT

3. Output Section

- LCD
- Buzzer
- GSM
- EV Charge

4. ESP32 Microcontroller

5. Software

There are five modules: Controller, RPS, Input, Software, and Output module. The power is supplied to the RPS module through an adapter. The adapter converts 230 volts AC to 12 volts DC and this 12 volts DC is given to the RPS module. The RPS module consists of a voltage regulator which converts the 12 volts DC into 3.3 volts DC, the capacitor is used to reduce noise and an LED is used to indicate whether the power is supplied or not. This 3.3 volts DC power supply is provided to each and every module.

There are three input modules in the EV Slot Reservation System. They are RTC, Keypad, and IoT module.

RTC (Real Time Clock) is used to keep track of the current time and date. It has three pins: Voltage, Ground, and Data. The Data pin is connected to the ESP32 controller. This helps in managing slot timings efficiently.

The Keypad is used for manual input from the user. It allows the user to enter booking details, authentication information, or select available slots. The data from the Keypad is directly given to the controller.

The IoT module is used to connect the system to the cloud. It enables remote access through a mobile application or website, allowing users to pre-book their slots in real-time. The data pin of the IoT module is connected to the ESP32 microcontroller.

There are four output modules in the system. They are LCD, Buzzer, GSM, and EV Charge.

LCD (Liquid Crystal Display) is used to display slot availability, booking status, and system feedback. It is a 16x2 display which shows real-time information to the user.

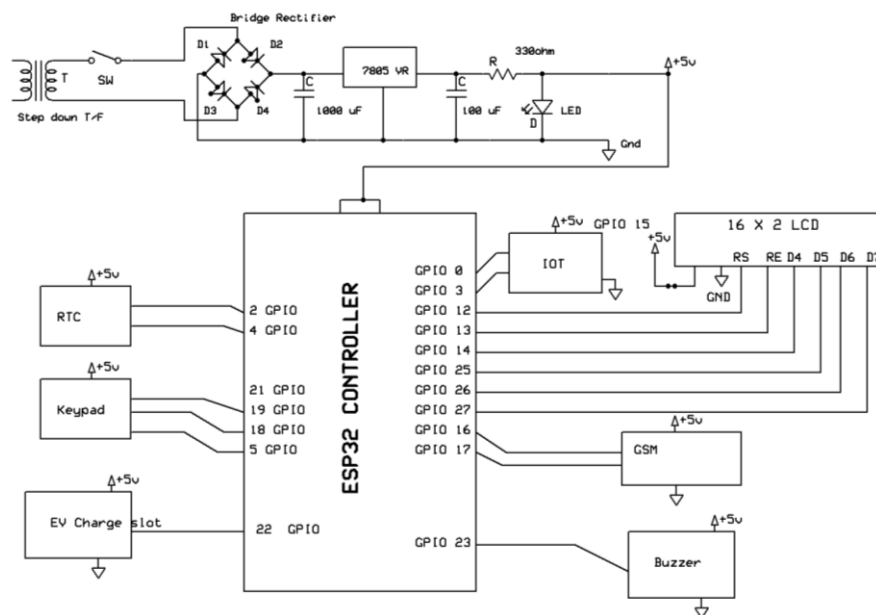
The Buzzer is used to provide sound alerts when booking is confirmed, canceled, or if any system notification needs to be given.

The GSM module is responsible for sending SMS notifications to the registered user's mobile number. It has three pins: Voltage, Ground, and Data. The data pin is connected to the ESP32, and the message is sent after successful slot booking.

EV Charge is connected to the charging unit via a relay circuit. Once the booking is confirmed and authenticated, the ESP32 triggers the relay, allowing power supply to the EV charging point. ESP32 microcontroller is the main component of the system. It collects data from the input modules, processes the information, and controls the output devices accordingly.

Software is used to program the ESP32 controller. It manages the real-time slot booking, data communication, and user interface through IoT platforms.

4.4 SCHEMATIC DIAGRAM



This is the pin diagram where all the hardware components are connected to the ESP32 microcontroller. The ESP32 microcontroller has 38 pins, including multiple GPIO pins functioning as both digital and analog input/output. It supports both Wi-Fi and Bluetooth communication internally. The system uses a Regulated Power Supply (RPS) consisting of a step-down transformer, bridge rectifier, capacitor, voltage regulator IC, resistors, and LED.

RPS circuit converts 230V AC to 12V DC, then regulates it to 3.3V DC, which is supplied to the ESP32 microcontroller and all the input/output modules.

The **16x2 LCD Monitor** is connected to the digital GPIO pins 4, 5, 18, 19, 21, and 22 of the ESP32 controller. It is used to display the current slot status, booking confirmation, and other system messages.

The **Keypad** is connected to digital pins 13, 12, 14, 27, 26, 25, 33, and 32 of the ESP32. It is used by the user to manually input booking details, password, or selection of charging slots. The **IoT (Wi-Fi) Module** functionality is internally supported by the ESP32 microcontroller itself through its built-in Wi-Fi capability. It allows communication with the cloud server for real-time booking and status updates.

The **RTC (Real Time Clock) Module** is connected to GPIO pins 23 and 19 via I2C communication protocol. It is used to maintain the real-time clock, even when power is off, ensuring accurate slot timing management.

The **Buzzer Alarm** is connected to GPIO pin 15. It is activated to notify the user regarding booking confirmation, slot unavailability, or any system alerts.

The **GSM Module** is connected to GPIO pins 16 and 17 (TX and RX pins). It is used for sending SMS notifications to the user's registered mobile number regarding booking details or confirmation messages.

The **EV Charging Relay Circuit** is connected to GPIO pin 2. The relay is triggered by the controller after successful booking and authentication, allowing the flow of current to the EV charging socket.

4.5 ADVANTAGES

- **Reduced Waiting Time:**

The system allows EV users to reserve charging slots in advance, eliminating the need to wait

in long queues and improving overall user convenience.

- **Efficient Resource Utilization:**

Charging station resources are optimally utilized through real-time slot allocation and monitoring, minimizing idle time and maximizing throughput.

- **Enhanced User Experience:**

A user-friendly interface comprising a custom 3-key keypad and LCD display ensures seamless navigation and booking for users, improving satisfaction.

- **Real-Time Monitoring:**

IoT integration enables real-time updates on slot availability, reservation status, and station utilization, providing transparency and improved station management.

- **Smart Energy Management:**

The system supports smart grid integration, distributing the charging load evenly across stations and reducing the risk of grid overloading.

- **Remote Access and Control:**

Administrators can remotely monitor, manage, and update slot statuses and user records via cloud-based platforms, facilitating ease of operation.

- **Data Security and Authentication:**

Secure booking mechanisms and user verification processes prevent unauthorized usage and fraudulent reservations, ensuring system reliability.

- **Scalability and Integration:**

The system is designed to be scalable, allowing integration with future smart city infrastructure and renewable energy sources.

4.6 APPLICATIONS

Public EV Charging Stations:

- Urban areas with high electric vehicle adoption rates can benefit from the implementation of the slot reservation system.
 - Users can pre-book charging slots, reducing congestion and enhancing the efficiency
- involves choosing an appropriate K for the test data. The performance is evaluated, and the best K is finalized for prediction.

Corporate EV Fleets:

- Businesses operating corporate EV fleets can utilize the system to manage and schedule charging sessions efficiently.
- This ensures reduced downtime, optimized fleet operations, and seamless energy management for corporate clients.

Residential Charging Hubs:

- Residential societies, apartment complexes, and gated communities can deploy the system to allow residents to reserve charging slots in advance.
- This prevents conflicts over charging station usage and improves convenience for residents.

Highway Charging Stations:

- Long-distance travelers can plan and reserve charging stops en route, eliminating range anxiety and ensuring a smooth travel experience.
- The system provides real-time updates on available stations along highways, enabling efficient trip planning.

Smart City Integration:

- The slot reservation system can be integrated with broader smart city platforms to facilitate centralized energy management.
- Integration with renewable energy sources and AI-driven demand forecasting can support sustainable urban mobility initiatives.

Electric Vehicle Showrooms and Service Centers:

- EV dealerships and service centers can implement the system to manage charging slots for test drives, maintenance, and after-sales services.
- This improves customer service and streamlines operations for businesses in the EV sector.

Commercial Parking Facilities:

- Parking lots and multi-level parking structures can install EV charging points equipped with the reservation system to attract EV users.
- Integration with payment systems enables automated billing and improves revenue management.

Government and Institutional Buildings:

- Public institutions, government offices, and educational campuses can deploy the system to promote sustainable transportation.
- Slot reservation ensures fair usage among staff and visitors.

CHAPTER – 5
EMBEDDED SYSTEMS

5.1 What are Embedded Systems:

An embedded system is a computer system designed to perform one or a few dedicated functions often with real-time computing constraints. It is embedded as part of a complete device often including hardware and mechanical parts. By contrast, a general-purpose computer, such as a personal computer (PC), is designed to be flexible and to meet a wide range of end-user needs. Embedded systems control many devices in common use today.

Embedded systems are controlled by one or more main processing cores that are typically either microcontrollers or digital signal processors (DSP). The key characteristic, however, is being dedicated to handle a particular task, which may require very powerful processors. For example, air traffic control systems may usefully be viewed as embedded, even though they involve mainframe computers and dedicated regional and national networks between airports and radar sites. (Each radar probably includes one or more embedded systems of its own.)

Since the embedded system is dedicated to specific tasks, design engineers can optimize it to reduce the size and cost of the product and increase the reliability and performance. Some embedded systems are mass-produced, benefiting from economies of scale.

Physically embedded systems range from portable devices such as digital watches and MP3 players, to large stationary installations like traffic lights, factory controllers, or the systems controlling nuclear power plants. Complexity varies from low, with a single microcontroller chip, to very high with multiple units, peripherals and networks mounted inside a large chassis or enclosure.

In general, "embedded system" is not a strictly definable term, as most systems have some element of extensibility or programmability. For example, handheld computers share some elements with embedded systems such as the operating systems and microprocessors which power them, but they allow different applications to be loaded and peripherals to be connected. Moreover, even systems which don't expose programmability as a primary feature generally need to support software updates. On a continuum from "general purpose" to "embedded", large application systems will have subcomponents at most points even if the system as a whole is "designed to perform one or a few dedicated functions", and is thus appropriate to call "embedded". A modern example of embedded system is shown in fig: 2.1.

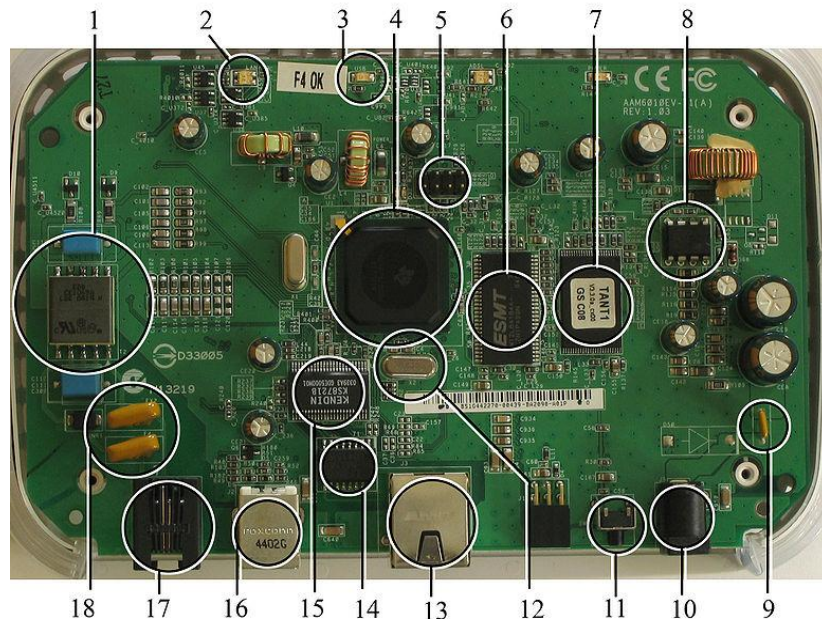


Fig 5.1: A modern example of embedded system

Labeled parts include microprocessor (4), RAM (6), flash memory (7). Embedded systems programming is not like normal PC programming. In many ways, programming for an embedded system is like programming PC 15 years ago. The hardware for the system is usually chosen to make the device as cheap as possible. Spending an extra dollar a unit in order to make things easier to program can cost millions. Hiring a programmer for an extra month is cheap in comparison. This means the programmer must make do with slow processors and low memory, while at the same time battling a need for efficiency not seen in most PC applications. Below is a list of issues specific to the embedded field.

5.1.1 History:

In the earliest years of computers in the 1930–40s, computers were sometimes dedicated to a single task, but were far too large and expensive for most kinds of tasks performed by embedded computers of today. Over time however, the concept of [programmable controllers](#) evolved from traditional [electromechanical](#) sequencers, via solid state devices, to the use of computer technology.

One of the first recognizably modern embedded systems was the [Apollo Guidance Computer](#), developed by [Charles Stark Draper](#) at the MIT Instrumentation Laboratory. At the

project's inception, the Apollo guidance computer was considered the riskiest item in the Apollo project as it employed the then newly developed monolithic integrated circuits to reduce the size and weight. An early mass-produced embedded system was the Autonetics D-17 guidance computer for the [Minuteman missile](#), released in 1961. It was built from [transistor logic](#) and had a [hard disk](#) for main memory. When the Minuteman II went into production in 1966, the D-17 was replaced with a new computer that was the first high-volume use of integrated circuits.

5.1.2 Tools:

Embedded development makes up a small fraction of total programming. There's also a large number of embedded architectures, unlike the PC world where 1 instruction set rules, and the UNIX world where there's only 3 or 4 major ones. This means that the tools are more expensive. It also means that they're lower featured, and less developed. On a major embedded project, at some point you will almost always find a compiler bug of some sort.

Debugging tools are another issue. Since you can't always run general programs on your embedded processor, you can't always run a debugger on it. This makes fixing your program difficult. Special hardware such as JTAG ports can overcome this issue in part. However, if you stop on a breakpoint when your system is controlling real world hardware (such as a motor), permanent equipment damage can occur. As a result, people doing embedded programming quickly become masters at using serial IO channels and error message style debugging.

5.1.3 Resources:

To save costs, embedded systems frequently have the cheapest processors that can do the job. This means your programs need to be written as efficiently as possible. When dealing with large data sets, issues like memory cache misses that never matter in PC programming can hurt you. Luckily, this won't happen too often- use reasonably efficient algorithms to start, and optimize only when necessary. Of course, normal profilers won't work well, due to the same reason debuggers don't work well.

Memory is also an issue. For the same cost savings reasons, embedded systems usually have the least memory they can get away with. That means their algorithms must be memory efficient (unlike in PC programs, you will frequently sacrifice processor time for memory, rather

than the reverse). It also means you can't afford to leak memory. Embedded applications generally use deterministic memory techniques and avoid the default "new" and "malloc" functions, so that leaks can be found and eliminated more easily. Other resources programmers expect may not even exist. For example, most embedded processors do not have hardware FPUs (Floating-Point Processing Unit). These resources either need to be emulated in software, or avoided altogether.

5.1.4 Real Time Issues:

Embedded systems frequently control hardware, and must be able to respond to them in real time. Failure to do so could cause inaccuracy in measurements, or even damage hardware such as motors. This is made even more difficult by the lack of resources available. Almost all embedded systems need to be able to prioritize some tasks over others, and to be able to put off/skip low priority tasks such as UI in favor of high priority tasks like hardware control.

5.2 Need For Embedded Systems:

The uses of embedded systems are virtually limitless, because every day new products are introduced to the market that utilizes embedded computers in novel ways. In recent years, hardware such as microprocessors, microcontrollers, and FPGA chips have become much cheaper. So when implementing a new form of control, it's wiser to just buy the generic chip and write your own custom software for it. Producing a custom-made chip to handle a particular task or set of tasks costs far more time and money. Many embedded computers even come with extensive libraries, so that "writing your own software" becomes a very trivial task indeed. From an implementation viewpoint, there is a major difference between a computer and an embedded system. Embedded systems are often required to provide Real-Time response. The main elements that make embedded systems unique are its reliability and ease in debugging.

5.2.1 Debugging:

Embedded debugging may be performed at different levels, depending on the facilities available. From simplest to most sophisticated they can be roughly grouped into the following areas:

- Interactive resident debugging, using the simple shell provided by the embedded operating system (e.g. Forth and Basic)
- External debugging using logging or serial port output to trace operation using either a monitor in flash or using a debug server like the Remedy Debugger which even works for heterogeneous multi core systems.
- An in-circuit debugger (ICD), a hardware device that connects to the microprocessor via a JTAG or Nexus interface. This allows the operation of the microprocessor to be controlled externally, but is typically restricted to specific debugging capabilities in the processor.
- An in-circuit emulator replaces the microprocessor with a simulated equivalent, providing full control over all aspects of the microprocessor.
- A complete emulator provides a simulation of all aspects of the hardware, allowing all of it to be controlled and modified and allowing debugging on a normal PC.
- Unless restricted to external debugging, the programmer can typically load and run software through the tools, view the code running in the processor, and start or stop its operation. The view of the code may be as assembly code or source-code.

Because an embedded system is often composed of a wide variety of elements, the debugging strategy may vary. For instance, debugging a software(and microprocessor) centric embedded system is different from debugging an embedded system where most of the processing is performed by peripherals (DSP, FPGA, co-processor). An increasing number of embedded systems today use more than one single processor core. A common problem with multi-core development is the proper synchronization of software execution. In such a case, the embedded system design may wish to check the data traffic on the busses between the processor cores, which requires very low-level debugging, at signal/bus level, with a logic analyzer, for instance.

5.2.2 Reliability:

Embedded systems often reside in machines that are expected to run continuously for years without errors and in some cases recover by them if an error occurs. Therefore the software is usually developed and tested more carefully than that for personal computers, and unreliable mechanical moving parts such as disk drives, switches or buttons are avoided.

Specific reliability issues may include:

- The system cannot safely be shut down for repair, or it is too inaccessible to repair. Examples include space systems, undersea cables, navigational beacons, bore-hole systems, and automobiles.
- The system must be kept running for safety reasons. "Limp modes" are less tolerable. Often backups are selected by an operator. Examples include aircraft navigation, reactor control systems, safety-critical chemical factory controls, train signals, engines on single-engine aircraft.
- The system will lose large amounts of money when shut down: Telephone switches, factory controls, bridge and elevator controls, funds transfer and market making, automated sales and service.

A variety of techniques are used, sometimes in combination, to recover from errors—both software bugs such as memory leaks, and also soft errors in the hardware:

- Watchdog timer that resets the computer unless the software periodically notifies the watchdog
- Subsystems with redundant spares that can be switched over to
- software "limp modes" that provide partial function
- Designing with a Trusted Computing Base (TCB) architecture[6] ensures a highly secure & reliable system environment
- An Embedded Hypervisor is able to provide secure encapsulation for any subsystem component, so that a compromised software component cannot interfere with other subsystems, or privileged-level system software. This encapsulation keeps faults from propagating from one subsystem to another, improving reliability. This may also allow a subsystem to be automatically shut down and restarted on fault detection.
- Immunity Aware Programming

5.3 Explanation of Embedded Systems:

5.3.1 Software Architecture:

There are several different types of software architecture in common use.

- Simple Control Loop:

In this design, the software simply has a loop. The loop calls subroutines, each of which manages a part of the hardware or software.

- **Interrupt Controlled System:**

Some embedded systems are predominantly interrupt controlled. This means that tasks performed by the system are triggered by different kinds of events. An interrupt could be generated for example by a timer in a predefined frequency, or by a serial port controller receiving a byte. These kinds of systems are used if event handlers need low latency and the event handlers are short and simple.

Usually these kinds of systems run a simple task in a main loop also, but this task is not very sensitive to unexpected delays. Sometimes the interrupt handler will add longer tasks to a queue structure. Later, after the interrupt handler has finished, these tasks are executed by the main loop. This method brings the system close to a multitasking kernel with discrete processes.

- **Cooperative Multitasking:**

A non-preemptive multitasking system is very similar to the simple control loop scheme, except that the loop is hidden in an API. The programmer defines a series of tasks, and each task gets its own environment to “run” in. When a task is idle, it calls an idle routine, usually called “pause”, “wait”, “yield”, “nop” (stands for no operation), etc. The advantages and disadvantages are very similar to the control loop, except that adding new software is easier, by simply writing a new task, or adding to the queue-interpreter.

- **Primitive Multitasking:**

In this type of system, a low-level piece of code switches between tasks or threads based on a timer (connected to an interrupt). This is the level at which the system is generally considered to have an "operating system" kernel. Depending on how much functionality is required, it introduces more or less of the complexities of managing multiple tasks running conceptually in parallel.

As any code can potentially damage the data of another task (except in larger systems using an MMU) programs must be carefully designed and tested, and access to shared data must be controlled by some synchronization strategy, such as message queues, semaphores or a non-blocking synchronization scheme.

Because of these complexities, it is common for organizations to buy a real-time operating system, allowing the application programmers to concentrate on device functionality

rather than operating system services, at least for large systems; smaller systems often cannot afford the overhead associated with a generic real time system, due to limitations regarding memory size, performance, and/or battery life.

• **Microkernels And Exokernels:**

A microkernel is a logical step up from a real-time OS. The usual arrangement is that the operating system kernel allocates memory and switches the CPU to different threads of execution. User mode processes implement major functions such as file systems, network interfaces, etc.

In general, microkernels succeed when the task switching and intertask communication is fast, and fail when they are slow. Exokernels communicate efficiently by normal subroutine calls. The hardware and all the software in the system are available to, and extensible by application programmers. Based on performance, functionality, requirement the embedded systems are divided into three categories:

5.3.2 Stand Alone Embedded System:

These systems takes the input in the form of electrical signals from transducers or commands from human beings such as pressing of a button etc., process them and produces desired output. This entire process of taking input, processing it and giving output is done in standalone mode. Such embedded systems comes under stand alone embedded systems

Eg: microwave oven, air conditioner etc..

5.3.3 Real-time embedded systems:

Embedded systems which are used to perform a specific task or operation in a specific time period those systems are called as real-time embedded systems. There are two types of real-time embedded systems.

• **Hard Real-time embedded systems:**

These embedded systems follow an absolute dead line time period i.e., if the tasking is not done in a particular time period then there is a cause of damage to the entire equipment.

Eg: consider a system in which we have to open a valve within 30 milliseconds. If this valve is not opened in 30 ms this may cause damage to the entire equipment. So in such cases we use

embedded systems for doing automatic operations.

- **Soft Real Time embedded systems:**

Eg: Consider a TV remote control system, if the remote control takes a few milliseconds delay it will not cause damage either to the TV or to the remote control. These systems which will not cause damage when they are not operated at considerable time period those systems comes under soft real-time embedded systems.

5.3.4 Network communication embedded systems:

A wide range network interfacing communication is provided by using embedded systems.

Eg:

- Consider a web camera that is connected to the computer with internet can be used to spread communication like sending pictures, images, videos etc., to another computer with internet connection throughout anywhere in the world.
- Consider a web camera that is connected at the door lock.

Whenever a person comes near the door, it captures the image of a person and sends to the desktop of your computer which is connected to internet. This gives an alerting message with image on to the desktop of your computer, and then you can open the door lock just by clicking the mouse. Fig: 2.2 show the network communications in embedded systems.



Fig 5.2: Network communication embedded systems

5.3.5 Different types of processing units:

The central processing unit (c.p.u) can be any one of the following microprocessor, microcontroller, digital signal processing.

- Among these Microcontroller is of low cost processor and one of the main advantage of microcontrollers is, the components such as memory, serial communication interfaces, analog to digital converters etc., all these are built on a single chip. The numbers of external components that are connected to it are very less according to the application.
- Microprocessors are more powerful than microcontrollers. They are used in major applications with a number of tasking requirements. But the microprocessor requires many external components like memory, serial communication, hard disk, input output ports etc., so the power consumption is also very high when compared to microcontrollers.
- Digital signal processing is used mainly for the applications that particularly involved with processing of signals

5.4 APPLICATIONS OF EMBEDDED SYSTEMS:

5.4.1 Consumer applications:

At home we use a number of embedded systems which include microwave oven, remote control, vcd players, dvd players, camera etc....



Fig5.3: Automatic coffee makes equipment

5.4.2 Office automation:

We use systems like fax machine, modem, printer etc...



Fig5.4: Fax machine



Fig5.5: Printing machine

5.4.3. Industrial automation:

Today a lot of industries are using embedded systems for process control. In industries we design the embedded systems to perform a specific operation like monitoring temperature, pressure, humidity ,voltage, current etc., and basing on these monitored levels we do control other devices, we can send information to a centralized monitoring station.



Fig 5.6
Robot

In critical industries where human presence is avoided there we can use robots which are programmed to do a specific operation.

5.4.4 Computer networking:

Embedded systems are used as bridges routers etc..



Fig5.7: Computer networking

5.4.5 Tele communications:

Cell phones, web cameras etc.



Fig5.8: Cell Phone

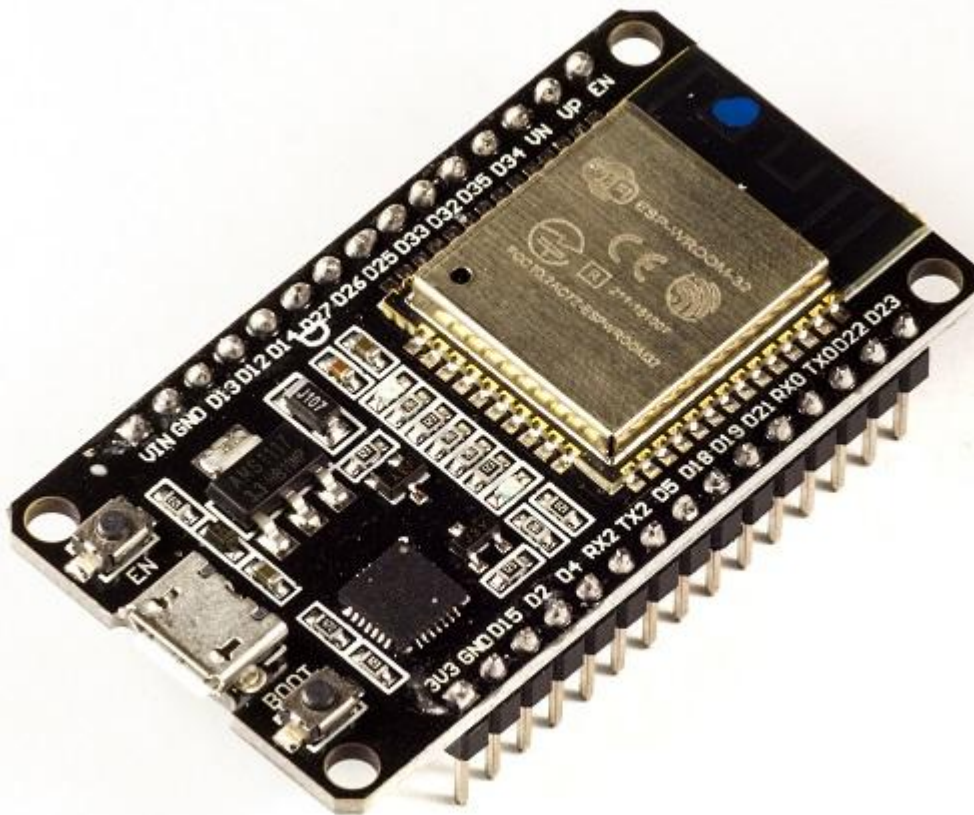


Fig5.9: Web camera

CHAPTER – 6
HARDWARE DESCRIPTION

6.1 ESP32 CONTROLLER

ESP32 is the SoC (System on Chip) microcontroller which has gained massive popularity recently. Whether the popularity of ESP32 grew because of the growth of IoT or whether IoT grew because of the introduction of ESP32 is debatable. If you know 10 people who have been part of the firmware development for any IoT device, chances are that 7–8 of them would have worked on ESP32 at some point.



Before we delve into the actual reasons for the popularity of ESP32, let's take a look at some of its important specifications. The specs listed below belong to the [ESP32 WROOM 32](#) variant.–

- Integrated Crystal– 40 MHz
- Module Interfaces– UART, SPI, I2C, PWM, ADC, DAC, GPIO, pulse counter, capacitive touch sensor
- Integrated SPI flash– 4 MB

- ROM– 448 KB (for booting and core functions)
- SRAM– 520 KB
- Integrated Connectivity Protocols– WiFi, Bluetooth, BLE
- On–chip sensor– Hall sensor
- Operating temperature range– –40 – 85 degrees Celsius
- Operating Voltage– 3.3V
- Operating Current– 80 mA (average)

With the above specifications in front of you, it is very easy to decipher the reasons for ESP32's popularity. Consider the requirements an IoT device would have from its microcontroller (μ C). If you've gone through the previous chapter, you'd have realized that the major operational blocks of any IoT device are sensing, processing, storage, and transmitting. Therefore, to begin with, the μ C should be able to interface with a variety of sensors. It should support all the common communication protocols required for sensor interface: UART, I2C, SPI. It should have ADC and pulse counting capabilities. ESP32 fulfills all of these requirements. On top of that, it also can interface with capacitive touch sensors. Therefore, most common sensors can interface seamlessly with ESP32.

Secondly, the μ C should be able to perform basic processing of the incoming sensor data, sometimes at high speeds, and have sufficient memory to store the data. ESP32 has a max operating frequency of 40 MHz, which is sufficiently high. It has two cores, allowing parallel processing, which is a further add-on. Finally, its 520 KB SRAM is sufficiently large for processing a large array of data onboard. Many popular processes and transforms, like FFT, peak detection, RMS calculation, etc. can be performed onboard ESP32. On the storage front, ESP32 goes a step ahead of the conventional microcontrollers and provides a file system within the flash. Out of the 4 MB of onboard flash, by default, 1.5 MB is reserved as SPIFFS (SPI Flash File System). Think of it as a mini–SD Card that lies within the chip itself. You can not only store data, but also text files, images, HTML and CSS files, and a lot more within SPIFFS. People have displayed beautiful WebPages on WiFi servers created using ESP32, by storing HTML files within SPIFFS.

Finally, for transmitting data, ESP32 has integrated WiFi and Bluetooth stacks, which have

proven to be a game-changer. No need to connect a separate module (like a GSM module or an LTE module) for testing cloud communication. Just have the ESP32 board and a running WiFi, and you can get started. ESP32 allows you to use WiFi in Access Point as well as Station Mode. While it supports TCP/IP, HTTP, MQTT, and other traditional communication protocols, it also supports HTTPS. Yep, you heard that right. It has a crypto-core or a crypto-accelerator, a dedicated piece of hardware whose job is to accelerate the encryption process. So you cannot only communicate with your web server, you can do so securely. BLE support is also critical for several applications. Of course, you can interface LTE or GSM or LoRa modules with ESP32. Therefore, on the 'transmitting data' front as well, ESP32 exceeds expectations. ESP32 is a chip that provides Wi-Fi and (in some models) Bluetooth connectivity for embedded devices – in other words, for IoT devices. While ESP32 is technically just the chip, the modules and development boards that contain this chip are often also referred to as “ESP32” by the manufacturer.

The original ESP32 chip had a single core Tensilica Xtensa LX6 microprocessor. The processor had a clock rate of over 240 MHz, which made for a relatively high data processing speed.

More recently, new models were added, including the ESP32-C and -S series, which include both single and dual core variations. These two series also rely on a Risc-V CPU model instead of Xtensa. Risc-V is similar to the ARM architecture, which is well-supported and well-known, but Risc-V is open source and easy to use. Specifically, Risc-V and ARM have good support from GNU compilers, while the Xtensa needed extra support and development to work with the compilers.

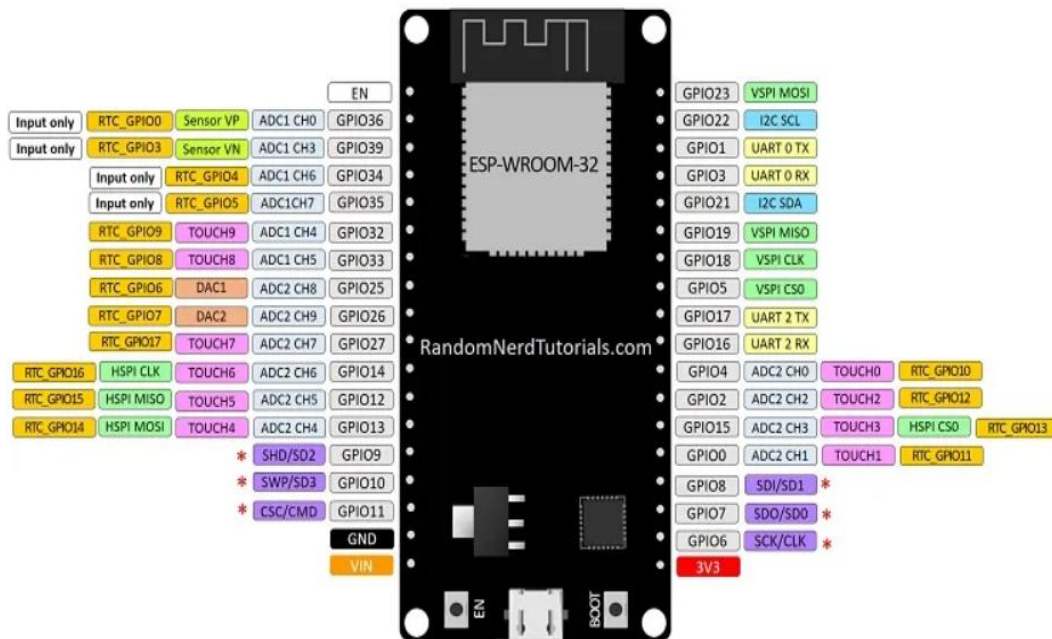
The newer models are available with combined Wi-Fi and Bluetooth connectivity, or just Wi-Fi connectivity. There are several different chip models available, including:

- ESP32-D0WDQ6 (and ESP32D0WD)
- ESP32-D2WD
- ESP32-S0WD
- System in package (SiP) – ESP32-PICO-D4
- ESP32 S series

- ESP32-C series

The ESP32 is most commonly used for mobile devices, wearable tech, and IoT applications, such as [Nabto](#) Edge. Moreover, since [Mongoose OS](#) introduced an [ESP32 IoT Starter Kit](#), the ESP32 has gained a reputation as the ultimate chip for hobbyists and IoT developers. It's suitable for commercial IoT, and its capabilities and resources have grown impressively over the past four years.

6.1.1 ESP32 features and specifications:



Here's a high-level summary of the features and specifications of the ESP32:

ESP-32	DESCRIPTION
Core	2
Architecture	32 bits
Clock	Tensilica Xtensa LX106 160-240MHz
WiFi	IEEE802.11 b/g/n
Bluetooth	Yes - classic & BLE
RAM	520KB
Flash	External QSPI - 16MB
GPIO	22
DAC	2
ADC	18
Interfaces	SPI-I2C-UART-I2S-CAN

- Processors** – The ESP32 uses a Tensilica Xtensa 32-bit LX6 microprocessor. This typically relies on a dual core architecture, with the exception of one module, the ESP32-S0WD, which uses a single-core system. The clock frequency reaches up to 240MHz and it performs up to 600 DMIPS (Dhrystone millions of instructions per second). Moreover, its low power consumption allows for analog to digital conversions as well as computation and level thresholds, even while the chip is in deep sleep mode.
- Wireless connectivity** – The ESP32 enables connectivity to integrated Wi-Fi through the 802.11 b/g/n/e/i/. Moreover, Bluetooth connectivity is made possible with the v4.2 BR/EDR, and the series also features Bluetooth low energy (BLE).
- Memory** – Internal memory for the ESP32 is as follows. ROM: 448 KB (for booting/core functions), SRAM: 520 KB (for data/instructions), RTC fast SRAM: 8 KB (for data storage/main CPU during boot from sleep mode), RTC slow SRAM: 8 KB (for co-processor access during sleep mode), and eFuse: 1 KiBit (256 bits used for the system (MAC address and chip configuration) and 768 bits reserved for customer applications). Moreover, some of the ESP32 chips, including the ESP32-D2WD and ESP32-PICO-D4, have internally

connected flash. See the respective internal flash memory for each chip in the [ESP32 Chips](#) section.

- **External flash and SRAM** – ESP32 supports up to four 16 MB external QSPI flashes and SRAMs with hardware encryption based on AES to protect developers' programs and data. It accesses the external QSPI flash and SRAM through high-speed caches.
- **Security** – The ESP32 supports all IEEE 802.11 standard security features, including WPA, WPA/WPA2 and WAPI. Moreover, ESP32 has a secure boot and flash encryption.

6.1.2 ESP32 functions:

ESP32 has many applications when it comes to IoT. Here are just some of the IoT functions the chip is used for:

- **Networking:** The module's Wi-Fi antenna and dual core enables embedded devices to connect to routers and transmit data.
- **Data processing:** Includes processing basic inputs from analog and digital sensors to far more complex calculations with an [RTOS](#) or non-OS software development kit (SDK). A non-OS SDK refers to one that is designed to run directly on the chip without a full operating system supporting it.
- **P2P connectivity:** Creates direct communication between different ESPs and other devices using [IoT P2P connectivity](#).
- **Web server:** Provides access to pages written in HTML or development languages.

ESP32 Applications:

The ESP32 modules are commonly found in the following IoT devices:

- [Smart industrial](#) devices, including programmable logic controllers (PLCs)
- [Smart medical](#) devices, including wearable health monitors
- [Smart energy](#) devices, including HVAC and thermostats
- [Smart security](#) devices, including surveillance cameras and smart locks

Chip versus modules versus development boards

The ESP32 is just the name of the chip. Device manufacturers and developers have three

different format choices, and the decision of which one to go with will depend on their individual circumstances:

- **ESP32 chip:** This is the bare-bones chip that is manufactured by Espressif. It comes unshielded, meaning there's no protective casing, and it can't be attached to a module or board without soldering. Therefore, most device manufacturers do not purchase just the chip, as this will add an additional layer of complexity to the production process.

- **ESP32 modules:** These are surface mountable modules that contain the chip. The modules are essentially small electrical components that can be attached to a circuit board. The benefit here is that you can easily mount these modules onto an MCU board. The chip is also usually shielded and pre-approved by the FCC, which means device manufacturers do not need to worry about adding additional steps to the production process to achieve FCC compliance regarding Wi-Fi shielding.

- **ESP32 development boards:** These are [IoT MCU](#) development boards that have the modules containing the ESP32 chip preinstalled. They are used by hobbyists, device manufacturers and developers to test and prototype IoT devices before entering mass production. There is a wide variety of makes and models of ESP32 development boards, produced by different manufacturers. Here are some important specs to consider when choosing a suitable IoT ESP32 development board:

- GPIO pins
- ADC pins
- Wi-Fi antennas
- LEDs
- Shielding
- Flash memory

*Many international markets require shielded Wi-Fi devices, as Wi-Fi produces a lot of radio frequency interference (RFI), and shielding minimizes this interference. This should, therefore,

be a key consideration for all developers and embedded device manufacturers.

6.2 Regulated Power Supply:

A regulated power supply converts unregulated AC (Alternating Current) to a constant

- DC (Direct Current). A regulated power supply is used to ensure that the output remains constant even if the input changes. A regulated DC power supply is also known as a linear power supply; it is an embedded circuit and consists of various blocks. The regulated power supply will accept an AC input and give a constant DC output. The figure below shows the block diagram of a typical regulated DC power supply.

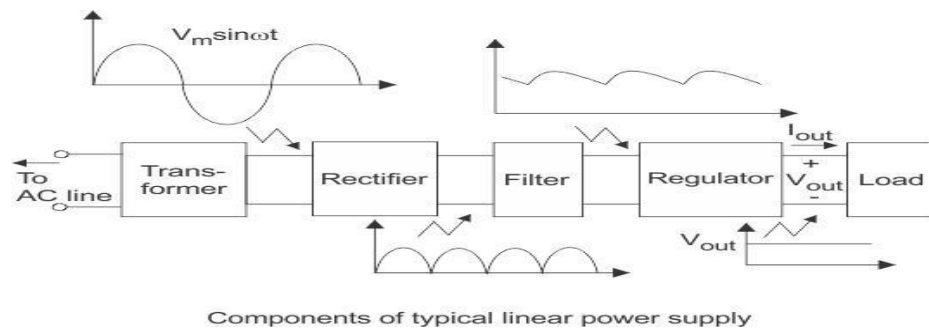


Figure 6.8 Block diagram of power supply

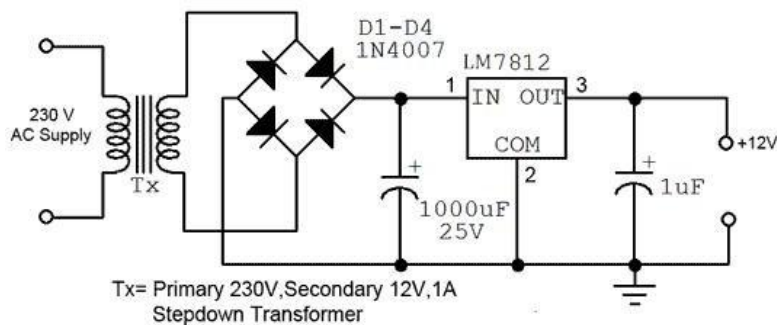


Figure 6.9 Power supply circuit diagram



Fig6.10 Regulated Power Supply

The basic circuit diagram of a regulated power supply (DC O/P) with led connected as load is shown in fig: 3.3.3.

REGULATED POWER SUPPLY

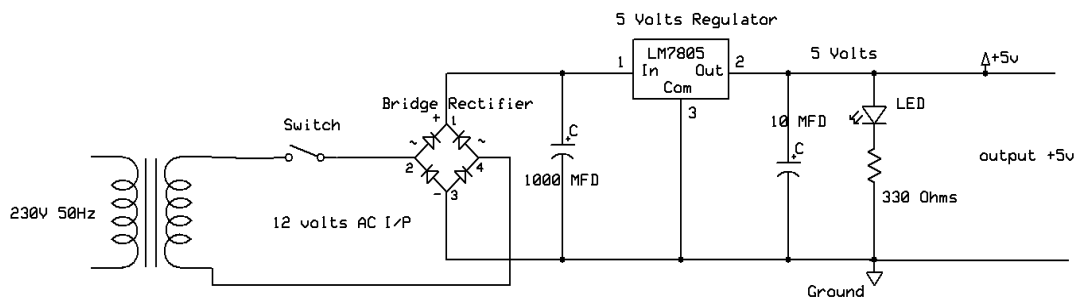


Fig 6.11 Circuit diagram of Regulated Power Supply with Led connection

The components mainly used in above figure are

- 230V AC MAINS
- TRANSFORMER
- BRIDGE RECTIFIER(DIODES)
- CAPACITOR
- VOLTAGE REGULATOR(IC 7805)

- RESISTOR
- LED(LIGHT EMITTING DIODE)

The detailed explanation of each and every component mentioned above is as follows:

Transformation: The process of transforming energy from one device to another is called transformation. For transforming energy we use transformers.

Step down transformer:

In case of step down transformer, Primary winding induces more flux than the secondary winding, and secondary winding is having less number of turns because of that it accepts less number of flux, and releases less amount of voltage.

6.2.1 Bridge full wave rectifier:

The Bridge rectifier circuit is shown in fig:3.8, which converts an ac voltage to dc voltage using both half cycles of the input ac voltage. The Bridge rectifier circuit is shown in the figure. The circuit has four diodes connected to form a bridge. The ac input voltage is applied to the diagonally opposite ends of the bridge. The load resistance is connected between the other two ends of the bridge.

For the positive half cycle of the input ac voltage, diodes D1 and D3 conduct, whereas diodes D2 and D4 remain in the OFF state. The conducting diodes will be in series with the load resistance R_L and hence the load current flows through R_L .

For the negative half cycle of the input ac voltage, diodes D2 and D4 conduct whereas, D1 and D3 remain OFF. The conducting diodes D2 and D4 will be in series with the load resistance R_L and hence the current flows through R_L in the same direction as in the previous half cycle. Thus a bi-directional wave is converted into a unidirectional wave.

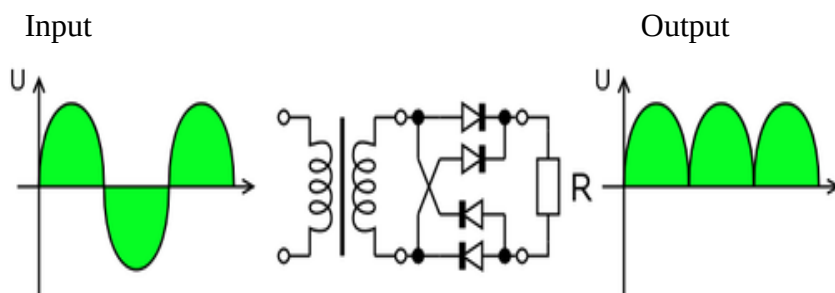


Fig 6.12 Bridge rectifier: a full-wave rectifier using 4 diodes

DB107:

Now -a -days Bridge rectifier is available in IC with a number of DB107. In our project we are using an IC in place of bridge rectifier. The picture of DB 107 is shown in fig: 3.3.8.



Fig 6.13. DB107

Capacitor Filtration:

The process of converting a pulsating direct current to a pure direct current using filters is called as filtration. Electronic filters are electronic circuits, which perform signal-processing functions, specifically to remove unwanted frequency components from the signal, to enhance wanted ones. The **Capacitor** or sometimes referred to as a Condenser is a passive device, and one which stores energy in the form of an electrostatic field which produces a potential (static voltage) across its plates. In its basic form a capacitor consists of two parallel conductive plates that are not connected but are electrically separated either by air or by an insulating material called the Dielectric. When a voltage is applied to these plates, a current flows charging up the plates with electrons giving one plate a positive charge and the other plate an equal and opposite negative charge this flow of electrons to the plates is known as the Charging Current and continues to flow until the voltage across the plates (and hence the capacitor) is equal to the applied voltage V_{cc} . At this point the capacitor is said to be fully charged and this is illustrated below. The construction of capacitor and an electrolytic capacitor are shown in figures 3.3.9 and 3.3.10 respectively.

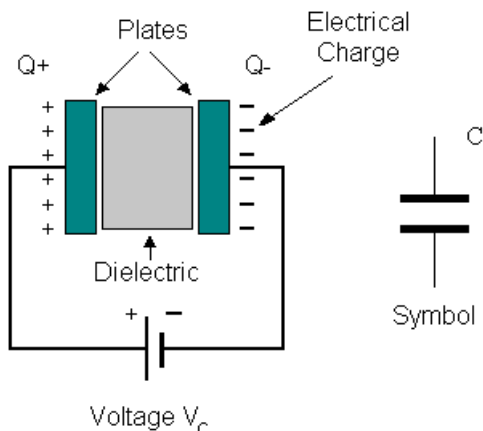


Fig 6.14 Construction Of a Capacitor

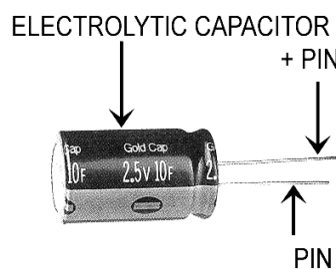


Fig 6.15 Electrolytic

6.2.2 Voltage Regulation:

The process of converting a varying voltage to a constant regulated voltage is called as regulation. For the process of regulation we use voltage regulators.

Voltage Regulator:

A voltage regulator (also called a ‘regulator’) with only three terminals appears to be a simple device, but it is in fact a very complex integrated circuit. It converts a varying input voltage into a constant ‘regulated’ output voltage. Voltage Regulators are available in a variety of outputs like 5V, 6V, 9V, 12V and 15V. The LM78XX series of voltage regulators are designed for positive input. For applications requiring negative input, the LM79XX series is used. Using a pair of ‘voltage-divider’ resistors can increase the output voltage of a regulator circuit.

It is not possible to obtain a voltage lower than the stated rating. You cannot use a 12V regulator to make a 5V power supply. Voltage regulators are very robust. These can withstand over-current draw due to short circuits and also over-heating. In both cases, the regulator will cut off before any damage occurs. The only way to destroy a regulator is to apply reverse voltage to its input. Reverse polarity destroys the regulator almost instantly. Fig: 3.3.11 shows voltage regulator.

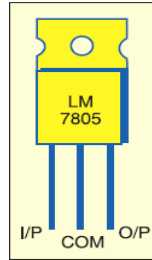


Fig 6.16. Voltage Regulator

6.2.3 Resistor:

A resistor is a two-terminal electronic component that produces a voltage across its terminals that is proportional to the electric current passing through it in accordance with Ohm's law:

$$V = IR$$

Resistors are elements of electrical networks and electronic circuits and are ubiquitous in most electronic equipment. Practical resistors can be made of various compounds and films, as well as resistance wire (wire made of a high-resistivity alloy, such as nickel/chrome).

The primary characteristics of a resistor are the resistance, the tolerance, maximum working voltage and the power rating. Other characteristics include temperature coefficient, noise, and inductance. Less well-known is critical resistance, the value below which power dissipation limits the maximum permitted current flow, and above which the limit is applied voltage. Critical resistance is determined by the design, materials and dimensions of the resistor.

dissipated by a resistor (or the equivalent resistance of a resistor network) is calculated using the following:

$$P = I^2 R = IV = \frac{V^2}{R}$$



Fig 6.18 Resistor

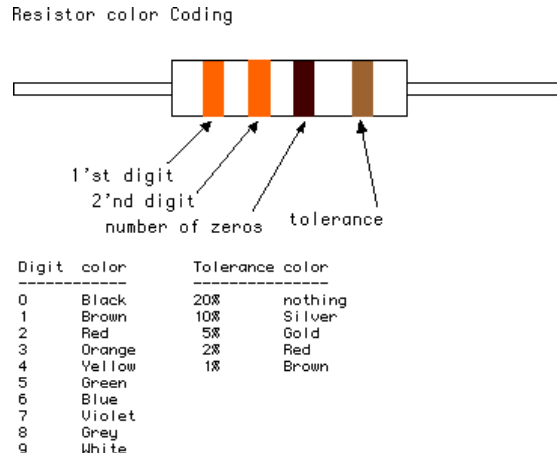


Fig 6.19 Color Bands In Resistor

LED:

A light-emitting diode (LED) is a semiconductor light source. LED's are used as indicator lamps in many devices, and are increasingly used for lighting. Introduced as a practical electronic component in 1962, early LED's emitted low-intensity red light, but modern versions are available across the visible, ultraviolet and infrared wavelengths, with very high brightness. The internal structure and parts of a led are shown in figures 6.20 and 6.21 respectively.

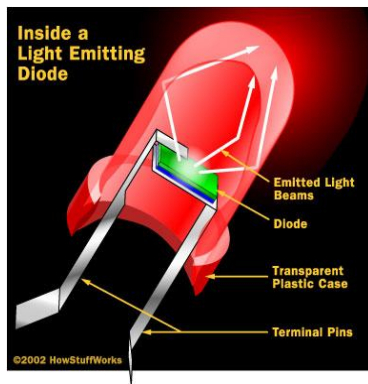


Fig.6.20 Inside a LED

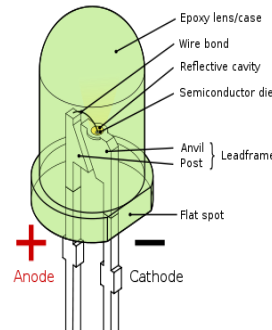


Fig 6.21. Parts of a LED

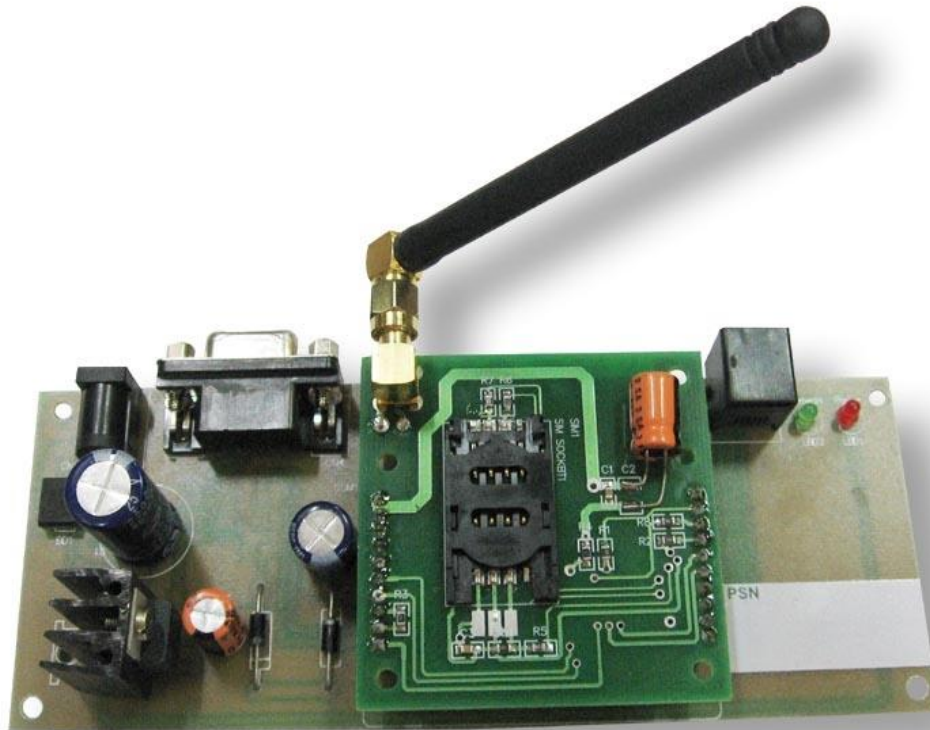
6.3 GSM (Global System for Mobile Communication)

6.3.1 Definition:

GSM, which stands for Global System for Mobile communications, reigns (important) as the world's most widely used cell phone technology. Cell phones use a cell phone service

carrier's GSM network by searching for cell phone towers in the nearby area. Global system for mobile communication (GSM) is a globally accepted standard for digital cellular communication.

GSM is the name of a standardization group established in 1982 to create a common European mobile telephone standard that would formulate specifications for a pan-European mobile cellular radio system operating at 900 MHz. It is estimated that many countries outside of Europe will join the GSM partnership



6.3.2 MODEM SPECIFICATIONS:

The SIM300 is a complete Tri-band GSM solution in a compact plug-in module.

Featuring an industry-standard interface, the SIM300 delivers GSM/GPRS900/1800/1900Mhz performance for voice, SMS, data and Fax in a small form factor and with low power consumption.

The leading features of SIM300 make it deal fir virtually unlimited application, such as WLL

applications (Fixed Cellular Terminal), M2M application, handheld devices and much more.

1. Tri-band GSM/GPRS module with a size of 40x33x2.85
2. Customized MMI and keypad/LCD support
3. An embedded powerful TCP/IP protocol stack
4. Based upon mature and field proven platform, backed up by our support service, from definition to design and production.

6.3.3 General Features:

- Tri-band GSM/GPRS900/1800/1900Mhz
 - GPRS multi-slot class 10
 - GPRS mobile station class –B
 - Complaint to GSM phase 2/2+
- i.-class 4(2W @900MHz)
- ii.-class 1(1W @/18001900MHz)
- Dimensions: 40x33x2.85 mm
 - Weight: 8gm
 - 7. Control via AT commands
 - (GSM 07.07, 07.05 and SIMCOM enhanced AT commands)
 - SIM application tool kit
 - supply voltage range 3.5.....4.5 v
 - Low power consumption
 - Normal operation temperature: -20 'C to +55 'C
 - Restricted operation temperature : -20 'C to -25 'C and +55 'C to +70 'C
 - storage temperature: -40 'C to +80 'C

Specifications for Fax:

Group 3 and class 1

Specifications for Data:

- GPRS class 10: max 85.6 kbps (downlink)
- PBCCH support
- coding schemes Cs 1,2,3,4
- CSD upto 14.4 kbps

- USSD
- Non transparent mode
- PPP-stack

Specifications for SMS via GSM/GPRS:

- Point to point MO and MT
- SMS cell broadcast
 - Text and PDU mode

Compatibility:

At cellular command interface

Specifications for voice:

1. Tricodec
 - Half rate (HR)
 - Full rate (FR)
 - Enhanced full rate (EFR)
2. Hands free operation
(Echo cancellation)

Drivers:

Microsoft windows mobile RIL driver
MUX driver

Interfaces:

- Interface to external SIM 3v 1.8v
- 60 pins board-to-board connector
- Two analog audio interfaces
- Keypad interfaces
- LCD interface
- RTC backup
- AT commands via serial interface
- Dual-Serial interfaces
- Antenna connector and antenna pad

Approvals:

- FTA
- Local type approval
- CE

Need of GSM:

The GSM study group aimed to provide the followings through the GSM:

- Improved spectrum efficiency.
- International roaming.
- Low-cost mobile sets and base stations (BS)
- High-quality speech
- Compatibility with Integrated Services Digital Network (ISDN) and other telephone company services.
- Support for new services.

6.4 Buzzer



Basically, the sound source of a piezoelectric sound component is a piezoelectric diaphragm. A piezoelectric diaphragm consists of a piezoelectric ceramic plate which has electrodes on both sides and a metal plate (brass or stainless steel, etc.). A piezoelectric ceramic plate is attached to a metal plate with adhesives. Applying D.C. voltage between electrodes of a piezoelectric diaphragm causes mechanical distortion due to the piezoelectric effect. For a misshaped piezoelectric element, the distortion of the piezoelectric element expands in a radial direction.

And the piezoelectric diaphragm bends toward the direction. The metal plate bonded to the piezoelectric element does not expand. Conversely, when the piezoelectric element shrinks, the piezoelectric diaphragm bends in the direction. Thus, when AC voltage is applied across electrodes, the bending is repeated, producing sound waves in the air. To interface a buzzer the standard

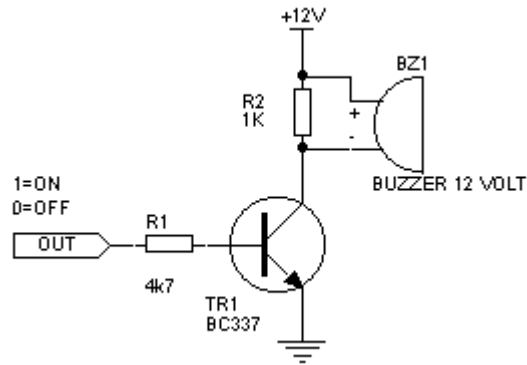
transistor interfacing circuit is used. Note that if a different power supply is used for the buzzer, the 0V rails of each power supply must be connected to provide a common reference. If a battery is used as the power supply, it is worth remembering that piezo sounders draw much less current than buzzers. Buzzers also just have one 'tone', whereas a piezo sounder is able to create sounds of many different tones.

To switch on buzzer -high 1

To switch off buzzer -low 1

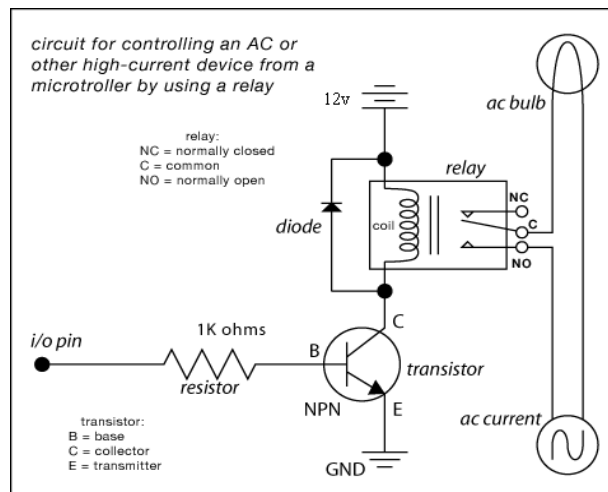
Notice (Handling) In Using Self Drive Method

- 1) When the piezoelectric buzzer is set to produce intermittent sounds, sound may be heard continuously even when the self drive circuit is turned ON / OFF at the "X" point shown in Fig. 9. This is because of the failure of turning off the feedback voltage.
- 2) Build a circuit of the piezoelectric sounder exactly as per the recommended circuit shown in the catalog of the transistor and circuit constants are designed to ensure stable oscillation of the piezoelectric sounder.
- 3) Design switching which ensures direct power switching.
- 4) The self drive circuit is already contained in the piezoelectric buzzer. So there is no need to prepare another circuit to drive the piezoelectric buzzer.
- 5) Rated voltage (3.0 to 20Vdc) must be maintained. Products which can operate with voltage higher than 20Vdc are also available.
- 6) Do not place resistors in series with the power source, as this may cause abnormal oscillation. If a resistor is essential to adjust sound pressure, place a capacitor (about 1 μ F) in parallel with the piezo buzzer.
- 7) Do not close the sound emitting hole on the front side of casing.
- 8) Carefully install the piezo buzzer so that no obstacle is placed within 15mm from the sound release hole on the front side of the casing.



6.5 Relay:

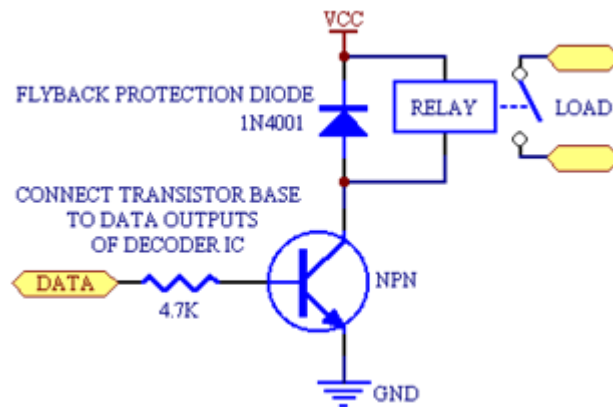
A relay is an electrically operated switch. Many relays use an electromagnet to operate a switching mechanism, but other operating principles are also used. Relays find applications where it is necessary to control a circuit by a low-power signal, or where several circuits must be controlled by one signal. The first relays were used in long distance telegraph circuits, repeating the signal coming in from one circuit and re-transmitting it to another. Relays found extensive use in telephone exchanges and early computers to perform logical operations. A type of relay that can handle the high power required to directly drive an electric motor is called a contactor. Solid-state relays control power circuits with no moving parts, instead using a semiconductor device triggered by light to perform switching. Relays with calibrated operating characteristics and sometimes multiple operating coils are used to protect electrical circuits from overload or faults; in modern electric power systems these functions are performed by digital instruments still called "protection relays".



6.5.1 Relay Driver:

The current needed to operate the relay coil is more than can be supplied by most chips (op. amps etc), so a transistor is usually needed, as shown in the diagram below.

Use BC109C or similar. A resistor of about 4k7 will probably be alright. The diode is needed to short circuit the high voltage “back emf” induced when current flowing through the coil is suddenly switched off.



RELAY DRIVER

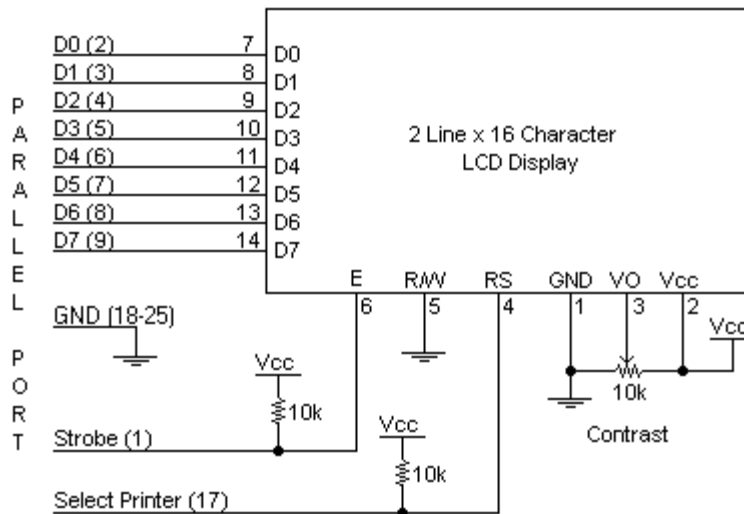
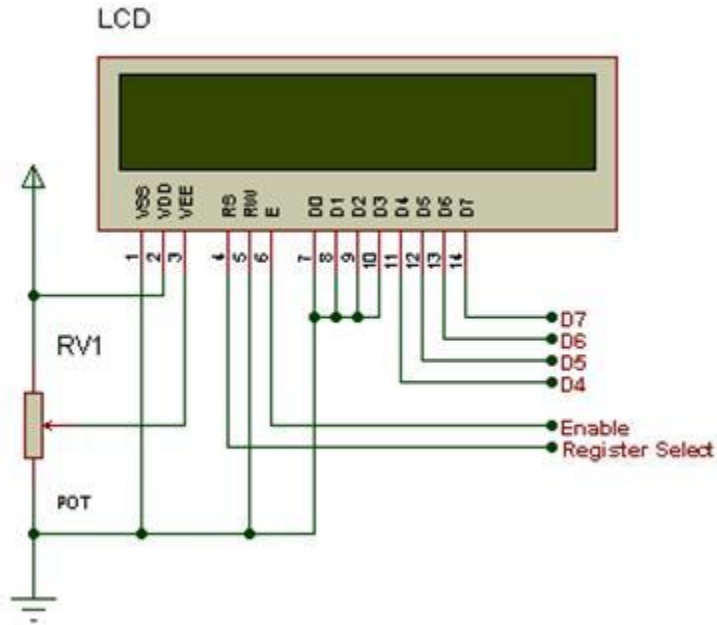
6.6 LCD DISPLAY

6.6.1 LCD Background:

One of the most common devices attached to a micro controller is an LCD display. Some of the most common LCD's connected to the many microcontrollers are 16x2 and 20x2 displays. This means 16 characters per line by 2 lines and 20 characters per line by 2 lines, respectively.

Basic 16x 2 Characters LCD

Figure 1: LCD Pin diagram



Circuit Description:

Above is the quite simple schematic. The LCD panel's Enable and Register Select is connected to the Control Port. The Control Port is an open collector / open drain output. While most Parallel Ports have internal pull-up resistors, there is a few which don't. Therefore by incorporating the two 10K external pull up resistors, the circuit is more portable for a wider range of computers, some of which may have no internal pull up resistors.

We make no effort to place the Data bus into reverse direction. Therefore we hard wire the R/W line of the LCD panel, into write mode. This will cause no bus conflicts on the data lines. As a result we cannot read back the LCD's internal Busy Flag which tells us if the LCD has accepted and finished processing the last instruction. This problem is overcome by inserting known delays into our program.

The 10k Potentiometer controls the contrast of the LCD panel. Nothing fancy here. As with all the examples, I've left the power supply out. We can use a bench power supply set to 5v or use an onboard +5 regulator. Remember a few de-coupling capacitors, especially if we have trouble with the circuit working properly.

6.6.2 SETB RW

Handling the EN control line:

As we mentioned above, the EN line is used to tell the LCD that we are ready for it to execute an instruction that we've prepared on the data bus and on the other control lines. Note that the EN line must be raised/ lowered before/after each instruction sent to the LCD regardless of whether that instruction is read or write text or instruction. In short, we must always manipulate EN when communicating with the LCD. EN is the LCD's way of knowing that we are talking to it. If we don't raise/lower EN, the LCD doesn't know we're talking to it on the other lines.

Thus, before we interact in any way with the LCD we will always bring the EN line low with the following instruction:

6.6.3 CLR EN

And once we've finished setting up our instruction with the other control lines and data bus lines, we'll always bring this line high:

SETB EN

The line must be left high for the amount of time required by the LCD as specified in its datasheet. This is normally on the order of about 250 nanoseconds, but check the datasheet. In the case of a typical microcontroller running at 12 MHz, an instruction requires 1.08 microseconds to execute so the EN line can be brought low the very next instruction. However, faster microcontrollers (such as the DS89C420 which executes an instruction in 90 nanoseconds given an 11.0592 MHz crystal) will require a number of NOPs to create a delay while EN is held high. The number of NOPs that must be inserted depends on the microcontroller we are using and the crystal we have selected.

The instruction is executed by the LCD at the moment the EN line is brought low with a final CLR EN instruction.

Checking the busy status of the LCD:

As previously mentioned, it takes a certain amount of time for each instruction to be executed by the LCD. The delay varies depending on the frequency of the crystal attached to the oscillator input of the LCD as well as the instruction which is being executed.

While it is possible to write code that waits for a specific amount of time to allow the LCD to execute instructions, this method of "waiting" is not very flexible. If the crystal frequency is changed, the software will need to be modified. A more robust method of programming is to use the "Get LCD Status" command to determine whether the LCD is still busy executing the last instruction received.

The "Get LCD Status" command will return to us two tidbits of information; the information that is useful to us right now is found in DB7. In summary, when we issue the "Get LCD Status" command the LCD will immediately raise DB7 if it's still busy executing a command or lower DB7 to indicate that the LCD is no longer occupied. Thus our program can query the LCD until DB7 goes low, indicating the LCD is no longer busy. At that point we are free to continue and send the next command.

6.6.4 Applications:

- Medical equipment
- Electronic test equipment
- Industrial machinery Interface
- Serial terminal
- Advertising system
- EPOS
- Restaurant ordering systems
- Gaming box
- Security systems
- R&D Test units
- Climatizing units
- PLC Interface

- Simulators
- Environmental monitoring
- Lab development
- Student projects
- Home automation
- PC external display
- HMI operator interface.

6.7 RTC(REAL TIME CLOCK):

A Real-Time Clock (RTC) module is **an integrated circuit (IC) that keeps track of the date and time, even when the main power is off, using a battery backup**. These modules are essential for applications needing accurate timestamps, such as embedded systems, alarm systems, and data logging.



6.7.1 Functions:

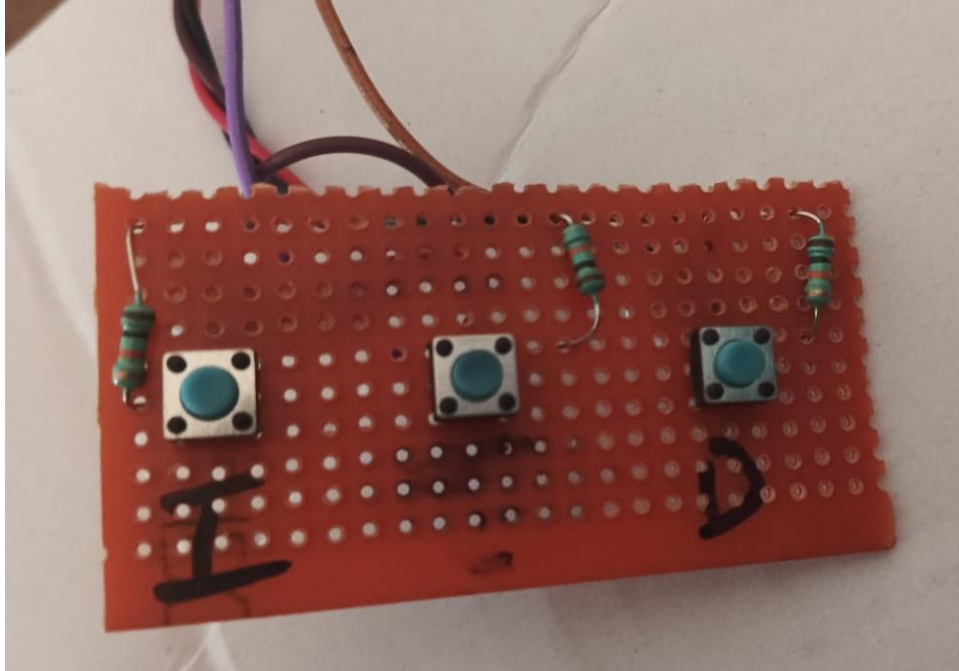
- **Timekeeping:** RTCs maintain accurate timekeeping, including seconds, minutes, hours, day, date, month, and year.

- **Battery Backup:** They use a small battery (like a coin cell) to keep the time running when the main power is disconnected.
- **Applications:**
 - **Embedded Systems:** For logging data with timestamps, scheduling events, and more.
 - **Computers/Laptops:** To maintain the system time even when powered off.
 - **Alarm Systems/Surveillance Cameras:** To accurately time events and logs.
 - **Industrial Products:** For critical time-sensitive operations.
- **Accuracy:** RTCs are designed for long-term accuracy, often using a 32.768 kHz crystal for timekeeping.
- **Low Power Consumption:** They are designed to consume very little power, even when running on battery backup.
- **Communication:** RTCs typically communicate with a microcontroller or other system using a serial interface like I2C or SPI.
- **Examples of RTC ICs:** DS1307, DS3231, DS1302

6.7.3 Key Features:

- **Accurate Timekeeping:** RTCs provide precise time and date information.
- **Battery Backup:** They maintain time even without main power.
- **Low Power:** Designed for minimal power consumption.
- **Compact Size:** RTCs are typically small and easy to integrate into various devices.
- **Versatile Applications:** Used in a wide range of applications, from embedded systems to computers.

6.8 KEYPAD:



The keypad module is an essential input device commonly used in embedded systems and microcontroller-based projects. It typically consists of a matrix of buttons or individual tactile switches arranged in a specific pattern to allow users to input numeric values, commands, or selections. Keypads are widely implemented in systems where simple, manual user interaction is required, such as ATMs, access control systems, and device control panels. They provide a reliable, low-cost, and straightforward interface for users to communicate with the electronic system. In our project, a custom-designed 3-key keypad module is used to facilitate user interaction. This keypad consists of three tactile push buttons mounted securely on a PCB board. Each button is connected to separate GPIO pins of the ESP32 microcontroller, allowing the system to detect and process the user's input effectively. The three keys are specifically labeled as 'I', 'D', and 'E'. The 'I' key stands for Increment, used to increase values such as slot numbers or options. The 'D' key represents Decrement, used to decrease values. The 'E' key stands for Enter, used to confirm the selection made by the user.

CHAPTER – 7
SOFTWARE DESCRIPTION

The Arduino Software (IDE) makes it easy to write code and upload it to the board offline. We recommend it for users with poor or no internet connection. This software can be used with any Arduino board. Here are currently two versions of the Arduino IDE, one is the IDE 2.0.0.

7.1 Arduino IDE – Compiler

here are currently two versions of the Arduino IDE, one is the IDE 1.x.x and the other is IDE 2.x. The IDE 2.x is new major release that is faster and even more powerful to the IDE 1.x.x. In addition to a more modern editor and a more responsive interface it includes advanced features to help users with their coding and debugging.

The following steps can guide you with using the offline IDE (you can choose either IDE 1.x.x or IDE 2.x):

1. Download and install the Arduino Software IDE:

- **Arduino IDE 1.x.x** ([Windows](#), [Mac OS](#), [Linux](#), [Portable IDE](#) for Windows and Linux, [ChromeOS](#)).
- [Arduino IDE 2.x](#)

2. Connect your Arduino board to your device.

3. Open the Arduino Software (IDE).

The Arduino Integrated Development Environment - or Arduino Software (IDE) - connects to the Arduino boards to upload programs and communicate with them. Programs written using Arduino Software (IDE) are called **sketches**. These sketches are written in the text editor and are saved with the file extension.ino.

Using the offline IDE 1.x.x

The editor contains the four main areas:

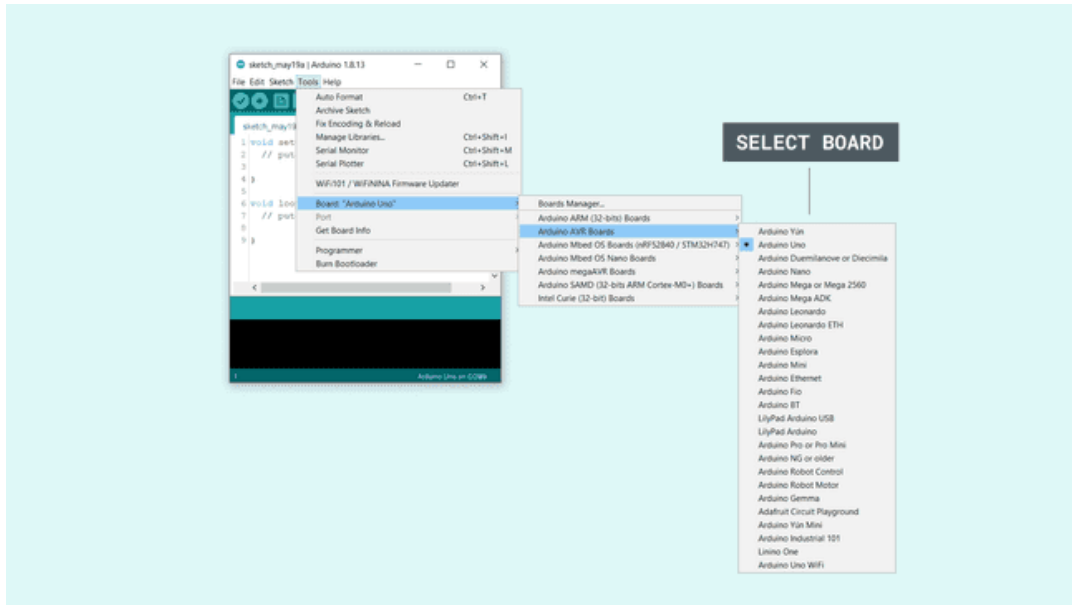
1. A **Toolbar with buttons** for common functions and a series of menus. The toolbar buttons allow you to verify and upload programs, create, open, and save sketches, and open the serial monitor.
2. The **message area**, gives feedback while saving and exporting and also displays errors.
3. The **text editor** for writing your code.
4. The **text console** displays text output by the Arduino Software (IDE), including complete error messages and other information. The bottom right-hand corner of .



The Arduino Software IDE

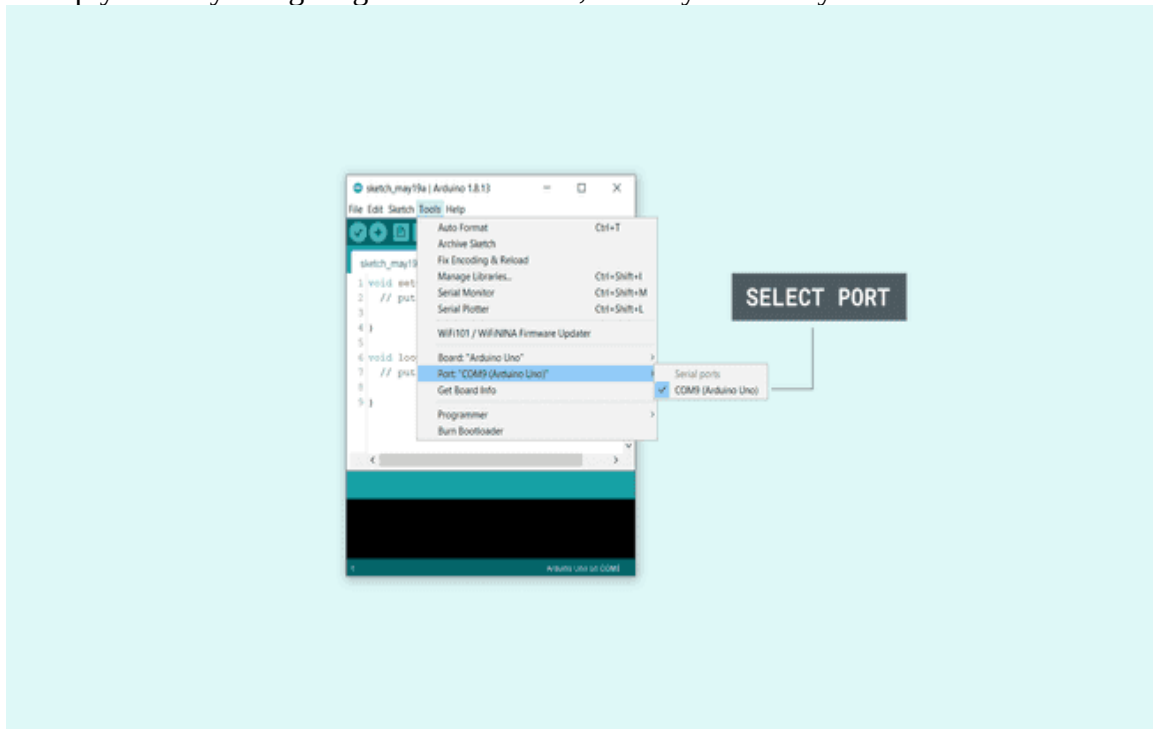
Now that you are all set up, **let's try to make your board blink!**

5. **Connect your Arduino** or Genuino board to your computer.
6. Now, you need to **select the right core & board**. This is done by navigating to **Tools > Board > Arduino AVR Boards > Board**. Make sure you select the board that you are using. If you cannot find your board, you can add it from **Tools > Board > Boards Manager**.

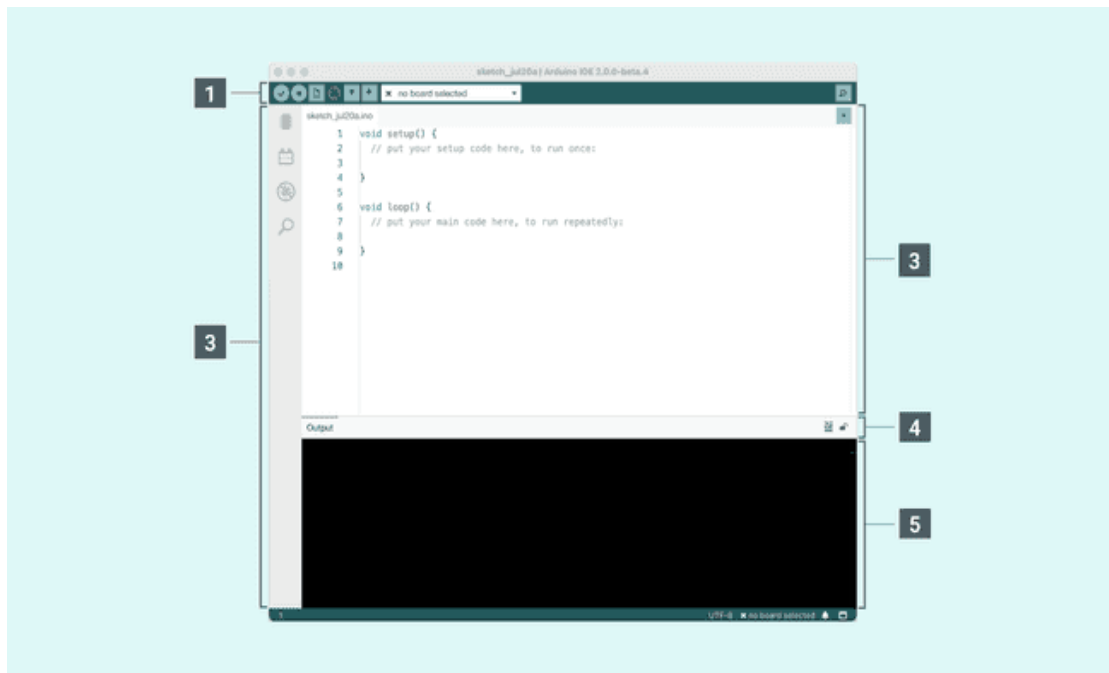


- Calls PerformanceMetrics() to evaluate and display the model's performance.

7. Now, let's make sure that your board is found by the computer, by **selecting the port**. This is simply done by navigating to **Tools > Port**, where you select your board from the list.



Selecting the port



The Arduino Software IDE

CHAPTER – 8
CODE

===== DECLARATION =====

```
#include <LiquidCrystal.h>
```

```
#include <stdio.h>
```

```
#include <Wire.h>
```

```
#include "RTCLib.h"
```

```
char daysOfTheWeek[7][12] = {"Sun", "Mon", "Tue", "Wed", "Thu", "Fri", "Sat"};
```

```
RTC_DS1307 rtc;
```

```
#include <WiFi.h>
```

```
#include <HTTPClient.h>
```

```
HTTPClient http;
```

```
const char *ssid = "iotserver";
```

```
const char *password = "iotserver123";
```

```
int httpResponseCode;
```

```
String servername = "http://projectsfactoryserver.in/storeddata.php?name=";
```

```
String accountname = "iot1207";
```

```
String field1 = "&s1=";
```

```
String field2 = "&s2=";
```

```
String field3 = "&s3=";
```

```
String field4 = "&s4=";
```

```
String field5 = "&s5=";
```

```
String payload="";
```

```
LiquidCrystal lcd(13, 12, 14, 27, 26, 25);
```

```
#define tx1 =16; // GSM
```

```
#define rx1=17;
```

```
#define tx2 =2; //RTC
```

```
#define rx2=4;
```

```
int swi = 19;
```

```
int swe = 18;
```

```
int swd = 5;
```

```
int buzzer = 23;
```

```
int relay = 22;
```

```
char pastnumber[11];
```

```
int secv=0,minv=0,hourv=0,dayv=0,monthv=0,yearv=0;
```

```
int secv1=0,minv1=0,hourv1=0;
int secv2=0,minv2=0,hourv2=0;
int secv3=0,minv3=0,hourv3=0;
int secv4=0,minv4=0,hourv4=0;
int year_c=0,month_c=0,day_c=0,hour_c=0,min_c=0,sec_c=0;
{
  lcd.setCursor(15,0);lcd.print("U");
  http.begin(servername + accountname + field1 + msg_string);
  httpResponseCode = http.GET();
  if(httpResponseCode>0)
  {
    payload="";
    //Serial.print("HTTP Response code: ");
    //Serial.println(httpResponseCode);
    payload = http.getString();
    //Serial.println(payload);
  }
  else
  {
    ;
    //Serial.print("Error code: ");
    //Serial.println(httpResponseCode);
  }
  delay(3000);
  lcd.setCursor(15,0);lcd.print(" ");
}
void beep()
{
  digitalWrite(buzzer, LOW);delay(2000);digitalWrite(buzzer, HIGH);
}
void okcheck()
{
  unsigned char rcr;
  do{
    rcr = Serial.read();
  }while(rcr != 'K');
}
void gsm_send(String gsm_string)
{
  lcd.setCursor(15,1);lcd.write('G');
  delay(5000);delay(4000);delay(4000);delay(4000);
  Serial.write("AT+CMGS=\"");
  Serial.write(pastnumber);
  Serial.write("\r\n"); delay(3000);
  Serial.print(gsm_string);
  Serial.write(0x1A);
}
```

```
delay(5000);delay(4000); delay(4000);delay(4000);
lcd.setCursor(15,1);lcd.write(' ');
}

void gsminit()
{
  Serial.write("AT\r\n");          okcheck();
  Serial.write("ATE0\r\n");        okcheck();
  Serial.write("AT+CMGF=1\r\n");    okcheck();
  Serial.write("AT+CNMI=1,2,0,0\r\n"); okcheck();
  Serial.write("AT+CSMP=17,167,0,0\r\n"); okcheck();

  lcd.clear();
  lcd.print("SEND MSG STORE");
  lcd.setCursor(0,1);
  lcd.print("MOBILE NUMBER");
  do{
    rcv = Serial.read();
  }while(rcv != '*');
  readSerial(pastnumber);

  lcd.clear();
  lcd.print(pastnumber);
  pastnumber[10]='\0';

  delay(4000); delay(4000);
  Serial.write("AT+CMGS=\"");
  Serial.write(pastnumber);
  Serial.write("\r\n"); delay(3000);
  Serial.write("Mobile no. registered\r\n");
  Serial.write(0x1A); delay(4000); delay(4000);
}

if(rtc.isrunning())
{
  lcd.print("RTC is NOT running!");
}
lcd.clear();
// rtc.adjust(DateTime(F(__DATE__), F(__TIME__))); //auto update from computer time
//rtc.adjust(DateTime(2024, 9, 20, 16, 28, 0)); // to set the time manually

gsminit();

lcd.clear();
```

```
//serialEvent();
}

===== logic =====
void loop()
{
  DateTime now = rtc.now();
  //yearv, monthv, dayv, hourv, minv, secv
  year_c = now.year();
  month_c = now.month();
  day_c = now.day();
  hour_c = now.hour();
  min_c = now.minute();
  sec_c = now.second();

  lcd.setCursor(0, 1);
  lcd.print(hour_c);
  lcd.print(':');
  lcd.print(min_c);
  lcd.print(':');
  lcd.print(sec_c);
  lcd.print(" ");

  lcd.setCursor(0, 0);
  lcd.print(daysOfTheWeek[now.dayOfTheWeek()]);
  lcd.print(" ");
  lcd.print(day_c);
  lcd.print('/');
  lcd.print(month_c);
  lcd.print('/');
  lcd.print(year_c);

  if(hour_c == hourv1 && min_c == minv1 && sec_c >=0 && sec_c <= 4)
  {
    lcd.clear();lcd.print("Pwd1 Sending.....");
    gsm_send("Password1:1234");
    lcd.clear();lcd.print("Pwd1 Send");
    for(int tip=0;tip<30;tip++)
    {
      lcd.setCursor(14,0);convertk(tip);
      if(digitalRead(swe) == LOW)
      {
        lcd.clear();lcd.print("Enter Password");
        while(digitalRead(swe) == LOW);delay(3000);
      }
    }
  }
}
```

```
lcd.setCursor(0,1);keypad();delay(1500);
if(strcmp(pwd,"1234") == 0)
{
lcd.clear();lcd.print("Correct Pwd");
digitalWrite(relay, HIGH);
gsm_send("Correct_Pwd_Charger_ON");
msg_string="";
msg_string="Correct_Pwd_Charger_ON";
iot_send();
digitalWrite(relay, LOW); goto mn1;
}
else
{
lcd.clear();lcd.print("Wrong Pwd");
beep();
gsm_send("Wrong_Pwd");
msg_string="";
msg_string="Wrong_Pwd";
iot_send();          goto mn1;
}
}
delay(1000);
}
mn1:
lcd.setCursor(14,0);lcd.print(" ");

if(hour_c == hourv2 && min_c == minv2 && sec_c >=0 && sec_c <= 4)
{
lcd.clear();lcd.print("Pwd2 Sending.....");
gsm_send("Password2:1235");
lcd.clear();lcd.print("Pwd2 Send");
for(int tip=0;tip<30;tip++)
{
lcd.setCursor(14,0);convertk(tip);
if(digitalRead(swe) == LOW)
{
lcd.clear();lcd.print("Enter Password");
while(digitalRead(swe) == LOW);delay(3000);

lcd.setCursor(0,1);keypad();delay(1500);
if(strcmp(pwd,"1235") == 0)
{
lcd.clear();lcd.print("Correct Pwd");
digitalWrite(relay, HIGH);
gsm_send("Correct_Pwd_Charger_ON");
```

```
msg_string="";
msg_string="Correct_Pwd_Charger_ON";
iot_send();
digitalWrite(relay, LOW); goto mn2;
}
else
{
lcd.clear();lcd.print("Wrong Pwd");
beep();
gsm_send("Wrong_Pwd");
msg_string="";
msg_string="Wrong_Pwd";
iot_send(); goto mn2;
}
}
delay(1000);
}
mn2:
lcd.setCursor(14,0);lcd.print(" ");
}

while(Serial.available() < 0)
{
char inChar = (char)Serial.read();//delay(5);
if(inChar == '@')
{sti=1;
}
if(sti == 1)
{
inputString += inChar;
}
if(inChar == '#')
{sti=0;
stringComplete = true;
}
}

if(stringComplete)
{
lcd.clear();

if(inputString[1] == 'A' || inputString[1] == 'a')
{
hourv1 = (((inputString[2]-48)*10) + (inputString[3]-48));
minv1 = (((inputString[5]-48)*10) + (inputString[6]-48));
secv1 = (((inputString[8]-48)*10) + (inputString[9]-48));
```

```
lcd.clear();lcd.write("Pwd-1 Configured");
}
if(inputString[1] == 'B' || inputString[1] == 'b')
{
    hourv2 = (((inputString[2]-48)*10) + (inputString[3]-48));
    minv2 = (((inputString[5]-48)*10) + (inputString[6]-48));
    secv2 = (((inputString[8]-48)*10) + (inputString[9]-48));

    lcd.clear();lcd.write("Pwd-2 Configured");
}

if(inputString[1] == 'C' || inputString[1] == 'c')
{
    hourv3 = (((inputString[2]-48)*10) + (inputString[3]-48));
    minv3 = (((inputString[5]-48)*10) + (inputString[6]-48));
    secv3 = (((inputString[8]-48)*10) + (inputString[9]-48));

    lcd.clear();lcd.write("Pwd-3 Configured");
}
if(inputString[1] == 'D' || inputString[1] == 'd')
{
    hourv4 = (((inputString[2]-48)*10) + (inputString[3]-48));
    minv4 = (((inputString[5]-48)*10) + (inputString[6]-48));
    secv4 = (((inputString[8]-48)*10) + (inputString[9]-48));

    lcd.clear();lcd.write("Pwd-4 Configured");
}

if(inputString[1] == 'T' || inputString[1] == 't')
{
    yearv = (((inputString[2]-48)*10) + (inputString[3]-48));
    monthv = (((inputString[5]-48)*10) + (inputString[6]-48));
    dayv = (((inputString[8]-48)*10) + (inputString[9]-48));
    lcd.clear();lcd.write("Time Configured");
}
if(inputString[1] == 'U' || inputString[1] == 'u')
{
    hourv = (((inputString[2]-48)*10) + (inputString[3]-48));
    minv = (((inputString[5]-48)*10) + (inputString[6]-48));
    secv = (((inputString[8]-48)*10) + (inputString[9]-48));
    rtc.adjust(DateTime(yearv, monthv, dayv, hourv, minv, secv));
    lcd.clear();lcd.write("D/T Configured");
}

delay(2000);
```

```
lcd.clear();
}
}

===== functions =====

void keypad()
{
while(1)
{
if(digitalRead(swi) == LOW)
{delay(700);
while(digitalRead(swi) == LOW);
cntt++;
if(cntt >= 9)
{
cntt=9;
}

//lcd(0xc0+pd); conv1(cntt);
lcd.setCursor(pd,1);convertk1(cntt);
}
if(digitalRead(swd) == LOW)
{delay(700);
while(digitalRead(swd) == LOW);
cntt--;
if(cntt <= 0)
{
cntt=0;
}
//lcd(0xc0+pd); conv1(cntt);
lcd.setCursor(pd,1);convertk1(cntt);
}

if(digitalRead(swe) == LOW)
{delay(700);
while(digitalRead(swe) == LOW);

pwd[pd] = (cntt+48);
pd++;

//lcd(0xc0+pd);
lcd.setCursor(pd,1);
```

```
cntt=0;
}
}
}

char modes='x';
String msg_string="";

void iot_send()

int sti=0;
String inputString = "";    // a string to hold incoming data
boolean stringComplete = false; // whether the string is complete

int sti2=0;
String inputString2 = "";    // a string to hold incoming data
boolean stringComplete2 = false; // whether the string is complete
String med1_string="";
String med2_string="";

float tempc=0,humc=0;
int val1 = 0,val2 = 0;
int sts1=0,sts2=0;
float vout=0;
int pd=0,cntt=0;
char pwd[5];
===== setup =====

void setup()
{
  Serial.begin(9600);//serialEvent();
  Serial2.begin(9600, SERIAL_8N1, RXD2, TXD2);

  pinMode(swi, INPUT_PULLUP);
  pinMode(swe, INPUT_PULLUP);
  pinMode(swd, INPUT_PULLUP);
  pinMode(buzzer, OUTPUT);pinMode(relay, OUTPUT);

  digitalWrite(buzzer, HIGH);digitalWrite(relay, LOW);

  lcd.begin(16, 2); lcd.print("EV Charge Slot");
  lcd.setCursor(0,1);lcd.print("Reservation System");
  delay(2500);

  WiFi.begin(ssid, password);
```

```
while(WiFi.status() != WL_CONNECTED)
{
  delay(500);
}
lcd.clear();lcd.write("WIFI Connected");
//Serial.println(WiFi.localIP());
delay(3000);

lcd.clear();
if(rtc.begin())
{
  lcd.print("Couldn't find RTC");
  while (1);
}
```

CHAPTER – 9
RESULTS

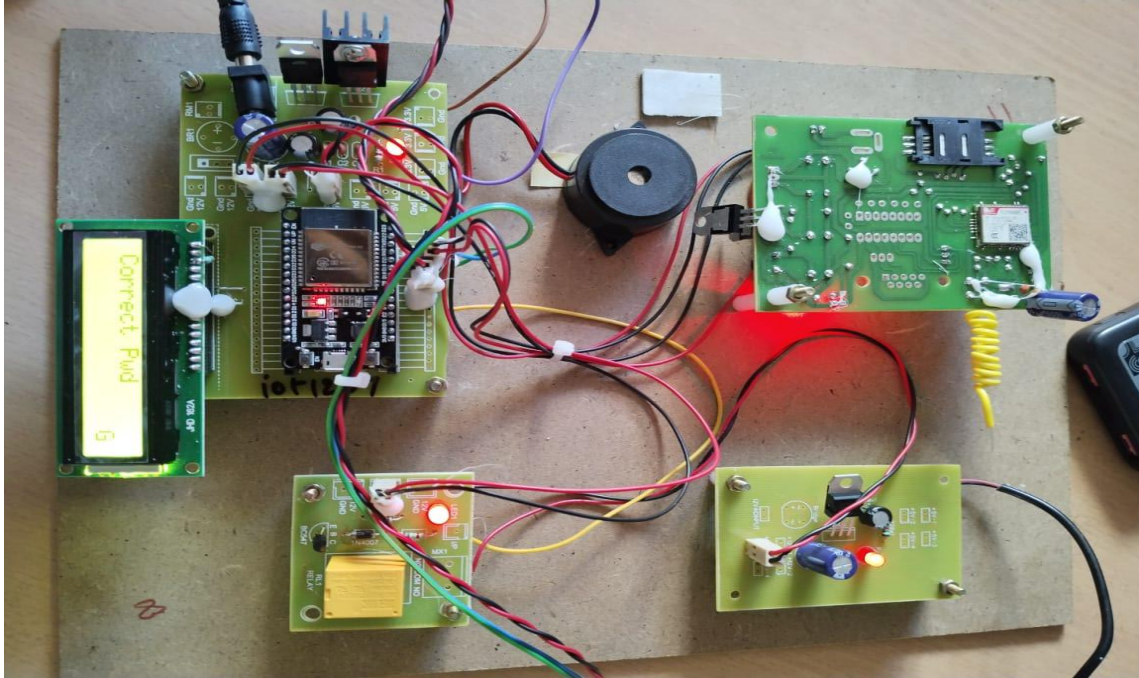


Figure 9.1 Hardware equipment

The hardware prototype of the **EV Slot Reservation System** consists of various integrated electronic components meticulously arranged to achieve the intended functionalities of reservation, authentication, and control. The core of the system is based on the **ESP32 microcontroller**, which manages all operations and communication between modules.

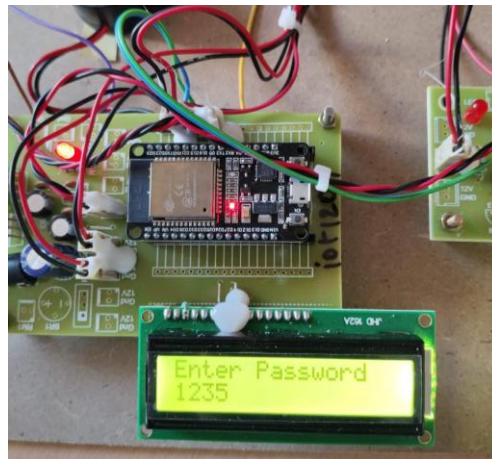


Figure 9.2 Password Entry

The **Password Entry Module** serves as a crucial element in the EV Slot Reservation System,

ensuring secure authentication of the user before granting access to the reserved charging slot. The system is designed with a **custom 3-key keypad interface**, which allows the user to input a unique password or reservation code.

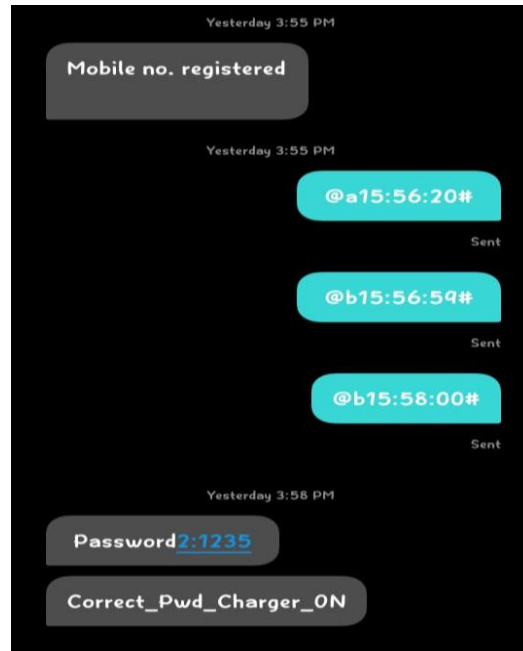


Figure 9.3 Commands

The EV Slot Reservation System integrates an efficient SMS-based communication protocol that enables remote registration, slot booking, and authentication using predefined commands. This mechanism enhances user convenience by allowing slot reservations and password verification via simple text messages. The following commands are employed within the system:

A screenshot of a web application interface. At the top, a teal banner contains the text 'Hello.. iot1207 Welcome to IOT Server' and a 'Logout' link. Below the banner are 'Refresh' and 'Switch to Graph View' buttons. The main content area shows 'Page 1 of 1' and a table with 3 columns: S.No, Charger Status, and Date. The table contains 9 rows of data.

S.No	Charger Status	Date
1	Correct_Pwd_Charger_ON	2025-03-21 15:19:09
2	Correct_Pwd_Charger_ON	2025-03-19 15:59:53
3	Correct_Pwd_Charger_ON	2025-03-19 15:45:27
4	Correct_Pwd_Charger_ON	2025-03-19 15:42:18
5	Correct_Pwd_Charger_ON	2025-03-19 15:29:15
6	Correct_Pwd_Charger_ON	2025-03-19 15:23:22
7	Correct_Pwd_Charger_ON	2025-03-19 15:21:24
8	Wrong_Pwd	2025-03-19 15:19:34
9	Correct_Pwd_Charger_ON	2025-03-19 15:17:24

Figure 9.4 Website for Remote Monitoring

The IoT Server acts as the central monitoring and control hub for the EV Slot Reservation System. It facilitates real-time data logging, charger status updates, and user authentication tracking, thereby enhancing the transparency and efficiency of the system. The interface is designed with simplicity and clarity to allow administrators to easily oversee charging activities and password validations

CHAPTER – 10
CONCLUSION

The Electric Vehicle (EV) Slot Reservation System presents a smart and efficient solution to address the growing challenges associated with EV charging infrastructure. With the rising adoption of electric vehicles worldwide, the demand for organized and scalable charging systems has become imperative. This project introduces a user-friendly and intelligent platform that enables EV users to reserve charging slots in advance through a simple interface, thereby eliminating long waiting periods and ensuring optimal utilization of available resources.

By integrating IoT-based hardware components, including the ESP32 microcontroller and a custom-designed 3-key keypad, the system offers real-time interaction and seamless communication between users and the charging station. The inclusion of an LCD display enhances the usability of the system by providing instant feedback during the reservation and charging process. Moreover, the implementation ensures secure slot allocation, remote monitoring capabilities, and data logging features to support transparent operation and auditing. This project not only contributes to the convenience and satisfaction of EV users but also supports the broader vision of smart city development and sustainable transportation. Through its modular design and scalability, the system holds the potential for integration with larger energy management networks and renewable energy sources in the future. Overall, the EV Slot Reservation System stands as a significant step toward building intelligent, eco-friendly, and user-centric infrastructure for the rapidly evolving electric mobility landscape.

REFERENCES

Zhang et al. (2021) developed an intelligent EV charging slot reservation system that leverages real-time traffic data and energy demand forecasting to optimize charging station usage. Their system dynamically analyzes traffic conditions and predicts energy demand to pre-allocate charging slots efficiently. The model improves overall charging efficiency, minimizes user waiting times, and reduces the occurrence of underutilized stations. By incorporating predictive analytics, the system ensures a seamless EV charging experience and contributes to the smooth operation of EV networks 【1】 .

Kumar et al. (2020) proposed an IoT-enabled EV slot reservation framework integrated with cloud computing technology. Their system facilitates real-time slot booking by connecting IoT sensors at charging stations to a centralized cloud database. This approach ensures that users receive immediate updates on slot availability, while also optimizing energy consumption based on usage patterns. The research demonstrates how cloud-based reservation systems can enhance user convenience, reduce operational costs, and improve resource utilization across charging networks 【2】 .

Lee and Park (2022) explored blockchain implementation in EV charging slot reservation systems to strengthen security and prevent fraudulent reservations. Their decentralized framework employs smart contracts to ensure transparent, tamper-proof transactions. This approach not only enhances data integrity but also protects against unauthorized slot bookings. The research shows that integrating blockchain technology into EV slot reservation systems effectively addresses trust and security concerns while maintaining operational efficiency 【3】 .

Chen et al. (2021) developed an AI-driven predictive model for charging slot allocation. Their system utilizes historical demand patterns combined with real-time data to anticipate peak usage times and allocate slots accordingly. By employing machine learning algorithms, the model continuously improves its predictions over time. The study highlights how predictive analytics can significantly enhance the efficiency of EV charging networks and reduce user wait times 【4】 .

Singh et al. (2019) presented a dynamic pricing model designed to regulate EV charging demand. The pricing strategy adjusts slot rates based on real-time demand fluctuations and congestion levels, encouraging users to utilize off-peak hours. This approach effectively balances the load on charging stations, promotes energy efficiency, and prevents grid overloading. The research demonstrates how dynamic pricing mechanisms can optimize resource distribution and improve user behavior **【5】** .

Patel et al. (2020) proposed a cloud-based scheduler for EV charging stations. The system manages slot reservations by processing real-time user requests alongside station availability data. The scheduler allocates charging slots dynamically, ensuring fair load distribution and preventing overutilization of specific stations. The research emphasizes how cloud integration and real-time scheduling can improve service reliability and user satisfaction **【6】** .

Wang et al. (2021) designed an optimization framework that incorporates power grid constraints into EV slot reservation systems. Their model takes into account available grid capacity, energy demand, and user requests to distribute charging slots effectively. This approach minimizes grid imbalance and improves station efficiency. The research suggests that such integrated frameworks are crucial to ensuring reliable, scalable EV charging networks **【7】** .

Li et al. (2022) integrated EV slot reservation systems into smart grids to balance electricity loads and prevent network congestion. Their system allows coordinated charging by synchronizing reservations with real-time grid conditions. This coordination reduces stress on power distribution systems, supports sustainable energy use, and enables better load management. The research underscores the importance of smart grid integration in modern EV infrastructure **【8】** .

Gomez et al. (2019) introduced a Vehicle-to-Grid (V2G)-enabled slot reservation system, where EVs not only consume energy but can also supply power back to the grid during peak hours. Their system supports bidirectional energy flow and incorporates pre-reserved slots to manage power transactions. This enhances grid stability and allows EV owners to benefit from energy trading. The study demonstrates how V2G technology can transform EVs into active participants in energy management **【9】**

Kumar and Singh (2020) developed an algorithm for real-time slot allocation based on user

preferences and station availability. Their system dynamically adjusts bookings, minimizing wait times and ensuring equitable distribution of charging slots. The research shows that real-time algorithms can improve the efficiency of charging station operations and enhances user experience 【10】 .

Smith et al. (2018) presented a mobile application that allows users to reserve EV charging slots in advance. The app provides real-time updates, estimated waiting times, payment integration, and navigation support. Their solution addresses range anxiety and user inconvenience by offering seamless access to charging infrastructure. The research highlights the role of mobile-based platforms in improving user engagement and station efficiency 【11】 .

Zhou and Fang (2021) proposed a multi-agent system to automate EV slot reservations. Independent agents represent user demands and station capacities, coordinating dynamically to distribute charging slots fairly. Their system enhances resource allocation efficiency and prevents over-concentration at specific stations. The research illustrates how multi-agent coordination can improve scalability and fairness in EV charging networks 【12】 .

Gupta et al. (2020) studied demand-side management techniques to optimize charging slot allocation. Their approach involves shifting charging loads to off-peak hours by offering incentives and dynamically adjusting slot availability. This reduces peak-hour congestion, prevents grid overloads, and lowers energy costs. The research emphasizes how demand-side management can contribute to sustainable EV infrastructure 【13】

Kim et al. (2019) employed predictive analytics to estimate future charging slot availability. Their model uses historical data trends and external factors, such as weather conditions, to forecast slot usage. This foresight allows users to plan their trips effectively, ensuring that charging stations are available when required. The research shows the benefits of incorporating predictive tools into reservation systems 【14】

Alam et al. (2021) implemented a machine learning-based EV slot reservation model that learns from user behaviors and station utilization patterns. defines its algorithms to minimize user wait times and improve efficiency. Their research demonstrates how adaptive machine learning models can optimize reservation processes in real-time 【15】 .

Hernandez et al. (2020) proposed an RFID-based system for automating EV charging slot bookings and access control. Users authenticate themselves using RFID tags, and the system manages reservations and station access accordingly. This contactless solution enhances security, speeds up the booking process, and prevents unauthorized station usage. The research highlights how RFID integration simplifies the user experience [16] .

Nakamura et al. (2021) integrated car-sharing services with EV slot reservation systems to ensure optimal utilization of charging infrastructure. Their approach allows shared EV fleets to access pre-designated slots, improving overall station throughput. The research suggests that integrating shared mobility services with slot management supports sustainable urban transportation solutions [17] .

Brown et al. (2022) analyzed how reservation systems impact the profitability of public EV charging stations. Their study shows that efficient reservation mechanisms increase station utilization rates, optimize revenue generation, and improve customer satisfaction. The research highlights the economic benefits of structured slot reservation systems [18] .

Liu et al. (2020) proposed a communication protocol between EV users and smart grids for efficient slot reservation and energy management. The protocol facilitates real-time data exchange, allowing dynamic load balancing and enhanced grid resilience. Their research emphasizes the importance of communication infrastructure in managing large-scale EV networks [19] .

Rossi et al. (2021) addressed challenges in EV slot reservation in urban areas characterized by high population density and limited infrastructure. They proposed solutions such as smart zoning, priority-based allocation, and real-time monitoring to alleviate congestion. Their study suggests innovative strategies to improve accessibility and manage demand in densely populated cities [20] .

PROGRAM OUTCOMES:

Engineering Graduates will be able to:

PO1. **Engineering Knowledge:** Apply the knowledge of mathematics, science, engineering fundamentals, and an engineering specialization to the solution of complex engineering problems.

PO2. **Problem Analysis:** Identify, formulate, review research literature, and analyze complex engineering problems reaching substantiated conclusions using first principles of mathematics, natural sciences, and engineering sciences.

PO3. **Design/development of solutions:** Design solutions for complex engineering problems and design system components or processes that meet the specified needs with appropriate consideration for the public health and safety, and the cultural, societal, and environmental considerations.

PO4. **Conduct investigations of complex problems:** Use research-based knowledge and research methods including design of experiments, analysis and interpretation of data, and synthesis of the information to provide valid conclusions.

PO5. **Modern tool usage:** Create, select, and apply appropriate techniques, resources, and modern engineering and IT tools including prediction and modelling to complex engineering activities with an understanding of the limitations.

PO6. **The engineer and society:** Apply reasoning informed by the contextual knowledge to assess societal, health, safety, legal and cultural issues and the consequent responsibilities relevant to the professional engineering practice.

PO7. **Environment and sustainability:** Understand the impact of the professional engineering solutions in societal and environmental contexts, and demonstrate the knowledge of, and need for sustainable development.

PO8. **Ethics:** Apply ethical principles and commit to professional ethics and responsibilities and norms of the engineering practice.

PO9. **Individual and team work:** Function effectively as an individual, and as a member or leader in diverse teams, and in multidisciplinary settings.

PO10. **Communication:** Communicate effectively on complex engineering activities with the engineering community and with society at large, such as, being able to comprehend and write effective reports and design documentation, make effective presentations, and give and receive clear instructions.

PO11. **Project management and finance:** Demonstrate knowledge and understanding of the engineering and management principles and apply these to one's own work, as a member and leader in a team, to manage projects and in multidisciplinary environments.

PO12. **Life-long learning:** Recognize the need for, and have the preparation and ability to engage in independent and life-long learning in the broadest context of technological change.